

LINUX_OTA_RESEARCH

Helix OTA — Linux OTA Support Research & Planning

Document ID: HELOTA-LINUX-001 **Version:** 1.1.0-draft **Status:** Active **Last Updated:** 2026-03-05 **Constitution Reference:** HelixConstitution v1 §1–§4, §7.1, §11.4.108 **Parent Roadmap:** VERSION_ROADMAP §7 (1.1.0-linux-support)

Table of Contents

- [1. Linux OTA Landscape Analysis](#)
 - [2. Ubuntu/Debian OTA Support](#)
 - [3. Fedora/rpm-ostree Support](#)
 - [4. Arch Linux Support](#)
 - [5. Generic Linux A/B Rootfs](#)
 - [6. OSAdapter Plugin Interface Design](#)
 - [7. New Submodules Required](#)
 - [8. Migration Path from Android-Only](#)
-

1. Linux OTA Landscape Analysis

1.1 How Different Linux Distributions Handle Updates

The Linux ecosystem presents a fundamentally different update paradigm from Android. Where Android enforces a single, centralized update mechanism (Google's `update_engine` with A/B partitions), Linux distributions have evolved multiple competing philosophies over three decades. Understanding these philosophies is essential for designing an OSAdapter interface that is both generic enough to encompass them all and specific enough to exploit each distribution's unique strengths.

Linux distributions cluster into three broad update strategies:

Package-based (traditional): The dominant model since the 1990s. Individual software packages (`.deb`, `.rpm`, `.pkg.tar.zst`) are independently versioned, dependency-resolved, and installed onto a mutable root filesystem. Updates are

granular — a single library can be updated without touching anything else. This granularity is both a strength (bandwidth efficiency, precision) and a weakness (dependency hell, partial-update inconsistency, difficult rollback).

Image-based (atomic): Emerging from the realization that traditional package management struggles with reproducibility and rollback in production/embedded contexts. The entire root filesystem is treated as an immutable, versioned unit. Updates replace the whole filesystem atomically. Rollback is trivial — just boot the previous image. This model sacrifices granularity for consistency and safety.

Hybrid: Some distributions blend both approaches, offering atomic base system updates with package layering on top. Fedora Silverblue, Ubuntu Core, and openSUSE MicroOS exemplify this pattern.

1.2 Package-Based Updates

APT (Debian/Ubuntu)

APT is the most widely deployed Linux package manager, serving Debian and its derivatives (Ubuntu, Linux Mint, Raspberry Pi OS, and hundreds more). It operates on a repository model: remote servers host Packages indices listing available `.deb` files, their versions, dependencies, and hashes. The client periodically fetches these indices (`apt update`), resolves a dependency graph, and downloads+installs the minimal set of packages needed to reach the desired state (`apt upgrade`).

OTA implications: APT updates are inherently non-atomic. If an `apt upgrade` is interrupted mid-transaction (power loss, network failure), the system can be left in an inconsistent state — some packages updated, others not, with potential dependency breakage. APT includes `dpkg --configure -a` for recovery, but this is not guaranteed to succeed. For OTA scenarios where reliability is paramount, raw APT is insufficient without additional scaffolding (snapshots, containers, or A/B partitioning).

Delta support: The `debdelta` tool can generate binary diffs between consecutive `.deb` package versions, reducing download sizes by 60–80% for minor version bumps. However, `debdelta` is largely unmaintained and not integrated into APT natively. The `apt-dater` tool provides centralized management of APT updates across multiple hosts, but it is designed for interactive sysadmin use, not automated OTA.

DNF (Fedora/RHEL/CentOS)

DNF (Dandified YUM) is the successor to YUM, used by Fedora and RHEL-family distributions. It operates on RPM repositories with a similar index-based model to APT. DNF's dependency resolver (`libsolv`) is more sophisticated than APT's, and it includes built-in delta RPM support.

OTA implications: DNF shares APT's non-atomic weakness. It does include `dnf history undo` for reverting transactions, but this is unreliable for complex updates and does not handle the case where the system is left unbootable.

Delta support: DNF natively supports presto deltas (deltarpm), which download only the binary diff between installed and available RPM versions. This is well-integrated and widely used in Fedora’s infrastructure.

pacman (Arch Linux)

pacman is Arch Linux’s package manager, known for its speed and simplicity. It uses `.pkg.tar.zst` packages (previously `.pkg.tar.xz`) with a flat repository structure. pacman does not perform automatic dependency resolution in the same way as APT/DNF — it expects the user to manage the full system upgrade as a unit (`pacman -Syu`), which aligns well with OTA principles.

OTA implications: Arch’s rolling release model means updates are frequent and unversioned — there is no concept of “Arch 15.0.1” to target. For OTA, this requires a custom repository with pinned package sets (snapshots). The `pacman -Syu` operation is not atomic, but Btrfs snapshots can provide atomicity.

Delta support: Arch’s pacman supports DeltaPackages — binary diffs between consecutive package versions. The `xdelta3` algorithm is used. However, delta support in official Arch repositories is inconsistent; it is more reliable in custom repositories (like those we would host for OTA).

zypper (openSUSE)

zypper is openSUSE’s package manager, built on libzypp. It offers the best transactional update support among traditional package managers through `transactional-update`, which uses Btrfs snapshots to provide atomic updates with instant rollback. This is the closest a traditional package manager comes to OTA-grade reliability.

1.3 Image-Based/Atomic Updates

OSTree

OSTree (libostree) is a content-addressed filesystem tree manager, often described as “git for operating system binaries.” It manages deployed trees as immutable, content-addressed directories. Each deployment is a checkout of a specific commit, and the system can atomically switch between deployments by updating the boot loader configuration.

How it works: 1. The OSTree repository stores filesystem trees as deduplicated content objects. 2. A “commit” references a specific tree state (analogous to a git commit). 3. A “deployment” is a checkout of a commit, mounted read-only at `/usr`. 4. `/etc` and `/var` are writable overlays per-deployment. 5. The bootloader (GRUB or systemd-boot) is configured to offer previous deployments as boot entries. 6. Static deltas are pre-computed binary diffs between commits, enabling efficient downloads.

OTA implications: OSTree is ideally suited for OTA. Updates are atomic (the new deployment is fully prepared before the bootloader is switched), rollback is instant (boot the previous deployment), and static deltas provide bandwidth-efficient

delivery. The trade-off is that the root filesystem is immutable — applications must be installed via container runtimes (Flatpak, Podman) or package layering (rpm-ostree).

rpm-ostree

rpm-ostree extends OSTree with the ability to layer RPM packages on top of the base OSTree tree. This provides the best of both worlds: atomic base system updates with the flexibility of RPM packages.

How it works: 1. The base system is an OSTree commit (e.g., Fedora Silverblue’s base image). 2. Users can layer additional RPMs with `rpm-ostree install <package>`. 3. These layered packages are recorded in a local overlay and merged into the deployment tree. 4. System updates (`rpm-ostree upgrade`) pull a new base commit and re-apply the layer on top. 5. If layering conflicts with the new base, the update is rejected with a clear error.

OTA implications: rpm-ostree is the strongest candidate for Fedora-based OTA. The atomic update model maps directly to Helix’s A/B partition philosophy. The main complexity is managing package layering across a fleet — each device may have different layered packages, making updates non-uniform.

bootc (Bootable Containers)

bootc is the next evolution of the OSTree/rpm-ostree lineage, introduced by Red Hat in 2023. It replaces OSTree’s custom transport with standard OCI container images. A bootc system boots from a container image, and updates are delivered by pulling a new container image (just like `podman pull`).

How it works: 1. The root filesystem is derived from an OCI container image. 2. `bootc upgrade` pulls the latest image and deploys it atomically. 3. `bootc rollback` switches to the previous image. 4. The container image can be stored in any OCI-compatible registry (Quay, Docker Hub, private registry).

OTA implications: bootc is the future direction for Fedora-based atomic updates. It simplifies the server-side infrastructure dramatically — instead of maintaining an OSTree repository, the Helix server only needs to push container images to a registry. However, bootc is still maturing (as of 2026) and may not be suitable for all deployment scenarios.

1.4 Hybrid Approaches

Several distributions combine atomic base updates with traditional package flexibility:

- **Ubuntu Core:** Uses snap packages for everything, with an A/B partition scheme for the base system. Snaps are individually updated, and the base system is atomically swapped.
- **openSUSE MicroOS:** Uses transactional-updates with Btrfs snapshots for the base system, with Flatpak/Distrobox for applications.
- **Fedora Silverblue:** Uses rpm-ostree for the base system with Flatpak for graphical applications and toolbox for development.

1.5 Comparison with Android's Approach

Android's OTA mechanism provides a useful reference point for understanding what Linux OTA must achieve:

Aspect	Android (A/B)	Linux Package-Based	Linux Image-Based
Atomicity	Full: A/B partition swap	None: partial update possible	Full: OSTree/bootc deployment swap
Rollback	Instant: boot old slot	Difficult: apt undo unreliable	Instant: boot previous deployment
Granularity	Full image only	Per-package	Full image + optional package layering
Delta efficiency	bsdiff between full images	per-package deltas (debdelta, deltarpm)	OSTree static deltas between commits
Verification	RSA signature on payload	GPG signature on repository metadata	GPG signature on OSTree commits
Bootloader	Android Boot Control HAL	GRUB/U-Boot/systemd-boot	GRUB/systemd-boot (OSTree-managed)
State management	update_engine state machine	No standard state machine	rpm-ostree transaction state
Server infrastructure	Custom OTA server	APT/RPM repository	OSTree repository or OCI registry

The key insight: Android's model is simpler because it controls the entire stack (OS, bootloader, update engine). Linux OTA must accommodate multiple bootloaders, multiple package managers, and multiple update philosophies — all through a single abstract interface.

2. Ubuntu/Debian OTA Support

2.1 APT-Based Update Mechanism

Ubuntu and Debian use APT as their primary package manager. For OTA purposes, we cannot rely on raw `apt upgrade` because it is not atomic, not auditable at the package level, and cannot be safely rolled back. Our approach must wrap APT with additional guarantees.

The core challenge: A fleet of Debian/Ubuntu devices may have diverged in their installed package sets. Device A may have `nginx` installed while Device B does not. An OTA update targeting “upgrade all packages” produces different results on each device. This non-uniformity violates the OTA principle that every device in a cohort should reach the same known state.

Our solution: We adopt a two-tier model:

1. **Base image updates** (A/B rootfs): The entire root filesystem is replaced atomically, identical to Android's A/B approach. This guarantees uniformity.
2. **Package-layer updates** (APT): For scenarios where base image updates are too coarse, we use pinned APT repositories with explicit package lists. The server defines exactly which packages at which versions should be installed, and the client enforces this.

2.2 Unattended-Upgrades Integration

Ubuntu ships with `unattended-upgrades`, a daemon that automatically installs security updates. For Helix OTA, we repurpose this infrastructure:

package aptadapter

```
// UnattendedUpgradesConfig configures the unattended-upgrades
// daemon
// to only install packages from the Helix OTA repository.
type UnattendedUpgradesConfig struct {
    // AllowedOrigins restricts upgrades to our custom repository
    AllowedOrigins []string `json:"allowed_origins"`
    // PackageWhitelist specifies exact packages to upgrade
    PackageWhitelist []string `json:"package_whitelist"`
    // PackageBlacklist prevents specific packages from being
    // upgraded
    PackageBlacklist []string `json:"package_blacklist"`
    // AutoFixBroken attempts to fix broken dependencies
    AutoFixBroken bool `json:"auto_fix_broken"`
    // MinimalSteps installs packages one at a time for safer
    // rollback
    MinimalSteps bool `json:"minimal_steps"`
    // OnCalendar specifies the systemd timer schedule
    OnCalendar string `json:"on_calendar"` // e.g., "0 2 * * *"
}
```

The Helix Linux client writes `/etc/apt/apt.conf.d/50helix-ota` with the appropriate configuration, then triggers `unattended-upgrades` with `--dry-run` first to validate the upgrade plan, and only then runs the actual upgrade.

2.3 A/B Rootfs Approach for Ubuntu Core Style

Ubuntu Core uses an A/B partition scheme that is remarkably similar to Android's. The root filesystem is mounted read-only, and updates write to the inactive partition. We adopt this model for standard Ubuntu/Debian as well.

Partition layout:

<code>/dev/mmcblk0p1</code>	<code>boot_a</code>	(EFI System Partition or extlinux boot)
<code>/dev/mmcblk0p2</code>	<code>boot_b</code>	(EFI System Partition or extlinux boot)
<code>/dev/mmcblk0p3</code>	<code>rootfs_a</code>	(ext4, read-only root filesystem)

```

/dev/mmcblk0p4 rootfs_b      (ext4, read-only root filesystem)
/dev/mmcblk0p5 userdata     (ext4, persistent user data, /var, /
etc overlay)
/dev/mmcblk0p6 swap         (swap partition)

```

The userdata partition is mounted at /var and contains the writable overlay (/etc, /home, /var/lib). This separation ensures that the root filesystem remains identical across all devices in a cohort, while device-specific state is preserved across updates.

2.4 Snap-Based Updates (Pros/Cons)

Ubuntu's Snap package system provides atomic, sandboxed application updates with automatic rollback. For Helix OTA, Snap is a tempting option because it solves the atomicity problem natively.

Pros: - Atomic updates with automatic rollback on failure - Delta updates built-in (snap deltas) - Confinement/sandboxing for security - Cross-distribution compatibility

Cons: - Snap Store is proprietary (Snapcraft); self-hosting a Snap Store requires the undocumented snap-store-proxy - Snap startup performance is poor due to squashfs mount overhead - Snap's confinement model is restrictive for system-level software - Snap's update cadence is controlled by Canonical, not the device operator - For embedded/IoT, Snap adds unnecessary complexity and resource overhead

Decision: We support Snap as an optional update channel for Ubuntu devices, but we do not make it the primary mechanism. The primary mechanism for Ubuntu/Debian is A/B rootfs with APT package overlay.

2.5 Custom APT Repository Management for OTA

Helix OTA must host its own APT repository to control exactly which packages are available to devices. This requires:

1. **Repository structure:** A standard Debian repository with dists/, pool/, and Packages/Release indices.
2. **GPG signing:** The repository metadata (Release, InRelease) must be signed with a GPG key that devices trust.
3. **Package pinning:** Devices must prefer the Helix repository over the distribution's default repository for managed packages.
4. **Version control:** The repository must contain exactly the packages that the current rollout specifies, preventing devices from pulling unapproved versions.

Server-side repository management:

package aptadapter

```

// APTRepositoryManager manages the Helix OTA APT repository.
// It is responsible for adding/removing packages, regenerating
// indices, and signing the repository metadata.
type APTRepositoryManager struct {
    repoDir    string        // Path to repository root on disk

```

```

    gpgSigner GPGSigner    // GPG signing implementation
    storage   storage.Provider // S3-compatible storage for
                           serving
}

// PublishPackages publishes a set of .deb packages to the
// repository
// and regenerates the indices.
func (m *APTRepositoryManager) PublishPackages(
    ctx context.Context,
    packages []DebPackage,
    distro string,    // e.g., "ubuntu"
    codename string, // e.g., "noble"
    component string, // e.g., "helix-ota"
) error {
    // 1. Copy .deb files to pool/
    for _, pkg := range packages {
        dest := filepath.Join(m.repoDir, "pool", component,
            pkg.Section, pkg.Filename)
        if err := copyFile(pkg.Path, dest); err != nil {
            return fmt.Errorf("copy package %s: %w", pkg.Name,
                err)
        }
    }

    // 2. Generate Packages index
    if err := m.generatePackagesIndex(distro, codename,
        component); err != nil {
        return fmt.Errorf("generate packages index: %w", err)
    }

    // 3. Generate Release file
    if err := m.generateReleaseFile(distro, codename); err != nil
    {
        return fmt.Errorf("generate release file: %w", err)
    }

    // 4. GPG-sign the Release file (produces Release.gpg and
    // InRelease)
    if err := m.signRelease(distro, codename); err != nil {
        return fmt.Errorf("sign release: %w", err)
    }

    // 5. Upload to S3-compatible storage
    if err := m.syncToStorage(ctx); err != nil {
        return fmt.Errorf("sync to storage: %w", err)
    }

    return nil
}

// DebPackage represents a .deb package to be published.
type DebPackage struct {
    Name string `json:"name"`

```



```

Version string `json:"version"`
Arch    string `json:"arch"`      // e.g., "amd64", "arm64"
Section string `json:"section"`   // e.g., "main", "utils"
Path    string `json:"path"`      // Local path to .deb file
Filename string `json:"filename"` // Filename in the pool
SHA256  string `json:"sha256"`
Size    int64  `json:"size"`
}

```

2.6 Delta Update Strategies

debdelta: The debdelta tool generates binary diffs between consecutive .deb package versions. A debdelta file is typically 30–60% the size of the full .deb. The client applies the delta to reconstruct the full .deb, then installs it normally.

Our approach: For APT-based updates, delta efficiency is less critical than for full-image updates because individual package updates are already small. However, for base rootfs A/B updates, we generate bsdiff deltas between the full rootfs images, identical to the Android delta strategy (reusing the helix-delta-gen submodule from v1.0.2).

package aptadapter

```

// DeltaStrategy determines how delta updates are generated for
// APT packages.
type DeltaStrategy int

const (
    // DeltaNone sends full .deb packages
    DeltaNone DeltaStrategy = iota
    // DeltaDebdelta uses debdelta binary diffs between package
    // versions
    DeltaDebdelta
    // DeltaZsync uses zsync for rsync-like efficient downloads
    DeltaZsync
)

// GenerateDebDelta creates a delta file between two .deb package
// versions.
func GenerateDebDelta(oldDeb, newDeb string) ([]byte, error) {
    // Use bsdiff algorithm (same as helix-delta-gen) on the .deb
    // files
    // This is more reliable than debdelta and is already
    // implemented
    return bsdiff.Diff(oldDeb, newDeb)
}

```

2.7 Implementation Plan with OSAdapter Interface

The Ubuntu/Debian adapter implements the OSAdapter interface with two operational modes: (1) A/B rootfs mode for full image updates, and (2) APT package mode for granular updates.

package aptadapter

```
import (  
    "context"  
    "fmt"  
    "os/exec"  
)
```

*// UbuntuAdapter implements OSAdapter for Ubuntu/Debian systems.
// It supports two modes: ABRootfs (full image) and APT (package-
level).*

```
type UbuntuAdapter struct {  
    mode          UbuntuUpdateMode  
    aptManager    *APTRepositoryManager  
    abManager     *ABRootfsManager  
    config        UbuntuAdapterConfig  
}
```

```
type UbuntuUpdateMode string
```

```
const (  
    ModeABRootfs UbuntuUpdateMode = "ab_rootfs"  
    ModeAPT       UbuntuUpdateMode = "apt"  
)
```

// UbuntuAdapterConfig holds configuration for the Ubuntu adapter.

```
type UbuntuAdapterConfig struct {  
    Mode          UbuntuUpdateMode `json:"mode" yaml:"mode"`  
    CurrentSlot   string           `json:"current_slot" yaml:"current_slot"`  
                                     // "_a" or "_b"  
    RootfsADevice string           `json:"rootfs_a_device" yaml:"rootfs_a_device"`  
                                     // e.g., "/dev/mmcblk0p3"  
    RootfsBDevice string           `json:"rootfs_b_device" yaml:"rootfs_b_device"`  
                                     // e.g., "/dev/mmcblk0p4"  
    APTRepoURL    string           `json:"apt_repo_url" yaml:"apt_repo_url"`  
                                     // e.g., "https://ota.example.com/apt"  
    APTRepoKeyURL string           `json:"apt_repo_key_url" yaml:"apt_repo_key_url"`  
                                     // GPG public key URL  
    APTDistro     string           `json:"apt_distro" yaml:"apt_distro"`  
                                     // e.g., "ubuntu"  
    APTCodename   string           `json:"apt_codename" yaml:"apt_codename"`  
                                     // e.g., "noble"  
    APTComponent string           `json:"apt_component" yaml:"apt_component"`  
                                     // e.g., "helix-ota"  
    PackageWhitelist []string      `json:"package_whitelist" yaml:"package_whitelist"`  
                                     // nil = all  
    Bootloader    string           `json:"bootloader" yaml:"bootloader"`  
                                     // "grub", "u-boot", "systemd-boot"  
    UserDataDevice string           `json:"userdata_device" yaml:"userdata_device"`  
                                     // e.g., "/dev/mmcblk0p5"  
}
```

// CheckForUpdate queries the current system state and determines

```

// whether an update is available.
func (a *UbuntuAdapter) CheckForUpdate(
    ctx context.Context,
    device Device,
) (*UpdateInfo, error) {
    switch a.mode {
    case ModeABRootfs:
        return a.abManager.CheckForUpdate(ctx, device)
    case ModeAPT:
        return a.aptCheckForUpdate(ctx, device)
    default:
        return nil, fmt.Errorf("unsupported mode: %s", a.mode)
    }
}

// ApplyUpdate triggers the Ubuntu-specific update mechanism.
func (a *UbuntuAdapter) ApplyUpdate(
    ctx context.Context,
    device Device,
    artifact Artifact,
) error {
    switch a.mode {
    case ModeABRootfs:
        return a.abManager.ApplyUpdate(ctx, device, artifact)
    case ModeAPT:
        return a.aptApplyUpdate(ctx, device, artifact)
    default:
        return fmt.Errorf("unsupported mode: %s", a.mode)
    }
}

// VerifyUpdate confirms the update was applied correctly.
func (a *UbuntuAdapter) VerifyUpdate(ctx context.Context, device
    Device) error {
    switch a.mode {
    case ModeABRootfs:
        return a.abManager.VerifyUpdate(ctx, device)
    case ModeAPT:
        return a.aptVerifyUpdate(ctx, device)
    default:
        return fmt.Errorf("unsupported mode: %s", a.mode)
    }
}

// Rollback reverts to the previous system state.
func (a *UbuntuAdapter) Rollback(ctx context.Context, device
    Device) error {
    switch a.mode {
    case ModeABRootfs:
        return a.abManager.Rollback(ctx, device)
    case ModeAPT:
        // APT rollback: restore from Btrfs snapshot or reinstall
        previous packages

```

```

        return a.aptrollback(ctx, device)
    default:
        return fmt.Errorf("unsupported mode: %s", a.mode)
    }
}

// ReportStatus returns the current update status.
func (a *UbuntuAdapter) ReportStatus(ctx context.Context, device
Device) (*UpdateStatus, error) {
    switch a.mode {
    case ModeABRootfs:
        return a.abManager.ReportStatus(ctx, device)
    case ModeAPT:
        return a.aptrReportStatus(ctx, device)
    default:
        return nil, fmt.Errorf("unsupported mode: %s", a.mode)
    }
}

// aptCheckForUpdate checks for available updates in APT mode.
func (a *UbuntuAdapter) aptCheckForUpdate(
    ctx context.Context,
    device Device,
) (*UpdateInfo, error) {
    // 1. Update package indices from Helix repository
    cmd := exec.CommandContext(ctx, "apt-get", "update", "-o",
        "Dir::Etc::sourcelist=/etc/apt/sources.list.d/helix-
        ota.list",
        "-o", "Dir::Etc::sourceparts=-")
    if output, err := cmd.CombinedOutput(); err != nil {
        return nil, fmt.Errorf("apt update failed: %s: %w",
            string(output), err)
    }

    // 2. Simulate upgrade to determine available updates
    cmd = exec.CommandContext(ctx, "apt-get", "upgrade", "--
        assume-no", "--dry-run")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return nil, fmt.Errorf("apt dry-run failed: %s: %w",
            string(output), err)
    }

    // 3. Parse dry-run output to determine if updates are
        available
    upgrades := parseAptDryRun(string(output))
    if len(upgrades) == 0 {
        return nil, nil // no update available
    }

    return &UpdateInfo{
        Version:      computeFingerprint(upgrades),
        Description: fmt.Sprintf("%d packages to upgrade",
            len(upgrades)),
    },

```

```

        SizeBytes: computeTotalSize(upgrades),
        Type: "apt_upgrade",
        Metadata: map[string]interface{}{
            "packages": upgrades,
        },
    }, nil
}

// aptApplyUpdate applies the APT upgrade.
func (a *UbuntuAdapter) aptApplyUpdate(
    ctx context.Context,
    device Device,
    artifact Artifact,
) error {
    args := []string{"upgrade", "--assume-yes", "--no-install-
        recommends"}
    if len(a.config.PackageWhitelist) > 0 {
        // Only upgrade whitelisted packages
        args = append(args, a.config.PackageWhitelist...)
    }
    cmd := exec.CommandContext(ctx, "apt-get", args...)
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("apt upgrade failed: %s: %w",
            string(output), err)
    }
    return nil
}

func (a *UbuntuAdapter) aptVerifyUpdate(ctx context.Context,
    device Device) error {
    // Verify no broken packages
    cmd := exec.CommandContext(ctx, "dpkg", "--audit")
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("dpkg audit found problems: %s: %w",
            string(output), err)
    }
    return nil
}

func (a *UbuntuAdapter) aptRollback(ctx context.Context, device
    Device) error {
    // APT rollback: reinstall previous package versions from
    // Helix repository
    // This requires the repository to retain previous package
    // versions
    // Alternative: use Btrfs snapshots if available
    return fmt.Errorf("APT rollback requires Btrfs snapshot
        support or previous package versions in repository")
}

func (a *UbuntuAdapter) aptReportStatus(ctx context.Context,
    device Device) (*UpdateStatus, error) {
    // Query dpkg for current package state
    cmd := exec.CommandContext(ctx, "dpkg", "--get-selections")

```

```

output, err := cmd.CombinedOutput()
if err != nil {
    return nil, fmt.Errorf("dpkg query failed: %w", err)
}
return &UpdateStatus{
    State: "IDLE",
    Metadata: map[string]interface{}{"dpkg_selections":
        string(output)},
}, nil
}

```

Implementation timeline:

Phase	Duration	Deliverable
Phase 1: A/B rootfs mode	3 weeks	Full Ubuntu A/B rootfs update with GRUB integration
Phase 2: APT package mode	2 weeks	APT repository management, package pinning, dry-run validation
Phase 3: Delta support	2 weeks	bsdiff deltas for rootfs, debdelta for packages
Phase 4: Testing & hardening	2 weeks	Mutation testing, chaos testing, E2E on real hardware

3. Fedora/rpm-ostree Support

3.1 rpm-ostree Atomic Update Mechanism

rpm-ostree is the update mechanism for Fedora Silverblue, Fedora Kinoite, and Red Hat's emerging Edge/IoT platforms. It combines OSTree's atomic deployment model with RPM's packaging format, providing the best atomic update story in the Linux ecosystem.

Update flow:

1. The client runs `rpm-ostree upgrade` which contacts the configured OSTree remote.
2. The new OSTree commit is downloaded (optionally as a static delta for efficiency).
3. A new deployment is prepared: the new tree is checked out, /etc is merged (three-way merge), and any layered RPMs are re-applied.
4. The bootloader entry for the new deployment is added as the default.
5. On next boot, the new deployment is active. The previous deployment remains available as a boot entry.
6. `rpm-ostree rollback` switches the bootloader default back to the previous deployment.

Key property: The update is fully transactional. If step 3 fails (e.g., layering conflict), the deployment is discarded and the running system is unaffected. If the new deployment fails at runtime, the administrator (or a health-check script) can trigger a rollback, and the system boots into the previous known-good state on the next reboot.

3.2 OSTree Repository Hosting

Helix OTA must host an OSTree repository to serve commits to Fedora devices. The repository can be either:

1. **Archive mode:** Stores all objects (files, dirtrees, commits) as individual compressed files. This is the standard mode for OSTree repositories that serve content over HTTP. It supports static delta generation.
2. **Bare mode:** Stores objects in their raw form for local checkouts. Not suitable for serving.

Our server maintains an archive-mode OSTree repository in the S3-compatible storage backend, served via a static file HTTP endpoint.

```
package ostreeadapter
```

```
import (
    "context"
    "fmt"
    "os/exec"
    "path/filepath"
)

// OSTreeRepositoryManager manages the server-side OSTree
// repository.
type OSTreeRepositoryManager struct {
    repoPath string // Local path to the archive-mode
    // repository
    storage storage.Provider // S3-compatible storage for serving
    gpgSigner GPGSigner
}

// InitializeRepository creates a new archive-mode OSTree
// repository.
func (m *OSTreeRepositoryManager) InitializeRepository(ctx
    context.Context) error {
    cmd := exec.CommandContext(ctx, "ostree", "init", "--repo",
        m.repoPath, "--mode", "archive")
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("ostree init failed: %s: %w",
            string(output), err)
    }
    return nil
}

// CommitTree commits a filesystem tree to the OSTree repository.
```

```

// The tree is typically built from a container image or a
// kickstart-based build.
func (m *OSTreeRepositoryManager) CommitTree(
    ctx context.Context,
    treePath string, // Path to the filesystem tree
    branch string,   // e.g., "fedora/40/x86_64/silverblue"
    subject string,  // Commit message subject
    body string,     // Commit message body
) (string, error) {
    // 1. Commit the tree
    cmd := exec.CommandContext(ctx, "ostree", "commit",
        "--repo", m.repoPath,
        "--tree=dir="+treePath,
        "--branch", branch,
        "--subject", subject,
        "--body", body,
        "--gpg-sign", m.gpgSigner.KeyID(),
    )
    output, err := cmd.CombinedOutput()
    if err != nil {
        return "", fmt.Errorf("ostree commit failed: %s: %w",
            string(output), err)
    }

    // 2. Extract the commit checksum
    commitHash := parseCommitHash(string(output))

    // 3. Sync to S3 for serving
    if err := m.syncToStorage(ctx); err != nil {
        return "", fmt.Errorf("sync to storage: %w", err)
    }

    return commitHash, nil
}

// ListCommits returns all commits on a branch.
func (m *OSTreeRepositoryManager) ListCommits(
    ctx context.Context,
    branch string,
) ([]OSTreeCommit, error) {
    cmd := exec.CommandContext(ctx, "ostree", "log",
        "--repo", m.repoPath,
        branch,
    )
    output, err := cmd.CombinedOutput()
    if err != nil {
        return nil, fmt.Errorf("ostree log failed: %s: %w",
            string(output), err)
    }
    return parseOSTreeLog(string(output)), nil
}

// OSTreeCommit represents an OSTree commit.

```



```

type OSTreeCommit struct {
    Hash      string `json:"hash"`
    Subject   string `json:"subject"`
    Body      string `json:"body"`
    Timestamp string `json:"timestamp"`
}

```

3.3 Static Delta Generation and Serving

Static deltas are pre-computed binary diffs between two OSTree commits. They dramatically reduce download size and server-side CPU load compared to pulling individual objects. Generating static deltas is a server-side operation.

```

package ostreeadapter

```

```

// GenerateStaticDelta creates a static delta between two commits.

```

```

func (m *OSTreeRepositoryManager) GenerateStaticDelta(
    ctx context.Context,
    fromCommit string,
    toCommit string,
) error {
    cmd := exec.CommandContext(ctx, "ostree", "static-delta",
        "generate",
        "--repo", m.repoPath,
        "--from", fromCommit,
        "--to", toCommit,
    )
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("static delta generation failed: %s: %w", string(output), err)
    }
}

```

```

// Sync deltas to S3

```

```

deltasDir := filepath.Join(m.repoPath, "deltas")
return m.storage.SyncDirectory(ctx, deltasDir, "ostree/
deltas/")

```

```

}

```

```

// GenerateFullDelta creates a "from-scratch" delta that does not
// require

```

```

// a source commit. This is used for initial deployments or when
// the

```

```

// source commit is unknown.

```

```

func (m *OSTreeRepositoryManager) GenerateFullDelta(
    ctx context.Context,
    toCommit string,
) error {
    cmd := exec.CommandContext(ctx, "ostree", "static-delta",
        "generate",
        "--repo", m.repoPath,
        "--to", toCommit,
        "--empty",
    )
}

```

```

    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("full delta generation failed: %s: %w",
            string(output), err)
    }
    return nil
}

```

3.4 bootc (Bootable Containers) as Future Direction

bootc simplifies the server-side infrastructure by replacing OSTree's custom repository format with standard OCI container images. Instead of managing an OSTree repository, the Helix server builds and pushes container images to a registry.

```

package ostreeadapter

```

```

// BootcAdapter implements OSAdapter using bootc (bootable
// containers).
// This is the recommended adapter for Fedora 41+ and future RHEL
// versions.

```

```

type BootcAdapter struct {
    registryURL string // OCI registry URL, e.g., "quay.io/helix-
        ota/fedora-silverblue"
    config      BootcAdapterConfig
}

```

```

type BootcAdapterConfig struct {
    RegistryURL    string `json:"registry_url"
        yaml:"registry_url"`
    ImageTag       string `json:"image_tag"
        yaml:"image_tag"` // e.g., "40.20260315"
    VerifySignature bool  `json:"verify_signature"
        yaml:"verify_signature"`
}

```

```

// CheckForUpdate checks for a newer image tag in the registry.

```

```

func (b *BootcAdapter) CheckForUpdate(
    ctx context.Context,
    device Device,
) (*UpdateInfo, error) {
    // Query the OCI registry for available tags
    // Compare with the device's current image digest
    cmd := exec.CommandContext(ctx, "bootc", "upgrade", "--check")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return nil, fmt.Errorf("bootc check failed: %s: %w",
            string(output), err)
    }
    return parseBootcCheckOutput(string(output)), nil
}

```

```

// ApplyUpdate triggers bootc upgrade.

```

```

func (b *BootcAdapter) ApplyUpdate(
    ctx context.Context,

```

```

        device Device,
        artifact Artifact,
    ) error {
        cmd := exec.CommandContext(ctx, "bootc", "upgrade")
        if output, err := cmd.CombinedOutput(); err != nil {
            return fmt.Errorf("bootc upgrade failed: %s: %w",
                string(output), err)
        }
        return nil
    }

    // Rollback triggers bootc rollback.
    func (b *BootcAdapter) Rollback(ctx context.Context, device
        Device) error {
        cmd := exec.CommandContext(ctx, "bootc", "rollback")
        if output, err := cmd.CombinedOutput(); err != nil {
            return fmt.Errorf("bootc rollback failed: %s: %w",
                string(output), err)
        }
        return nil
    }

    // VerifyUpdate confirms the new image is running.
    func (b *BootcAdapter) VerifyUpdate(ctx context.Context, device
        Device) error {
        cmd := exec.CommandContext(ctx, "bootc", "status")
        output, err := cmd.CombinedOutput()
        if err != nil {
            return fmt.Errorf("bootc status failed: %s: %w",
                string(output), err)
        }
        // Parse output to verify we're running the expected image
        return verifyBootcStatus(string(output), device.TargetVersion)
    }

    // ReportStatus returns the current bootc status.
    func (b *BootcAdapter) ReportStatus(ctx context.Context, device
        Device) (*UpdateStatus, error) {
        cmd := exec.CommandContext(ctx, "bootc", "status", "--json")
        output, err := cmd.CombinedOutput()
        if err != nil {
            return nil, fmt.Errorf("bootc status failed: %w", err)
        }
        return parseBootcStatusJSON(string(output))
    }
}

```

3.5 Implementation Plan

Phase	Duration	Deliverable
Phase 1: rpm-ostree adapter	3 weeks	rpm-ostree upgrade/rollback adapter with OSTree remote configuration

Phase	Duration	Deliverable
Phase 2: OSTree repository hosting	2 weeks	Server-side OSTree repository management with GPG signing
Phase 3: Static delta generation	2 weeks	Automatic delta generation on commit, serving via S3
Phase 4: bootc adapter	2 weeks	bootc-based adapter for Fedora 41+ devices
Phase 5: Testing	2 weeks	Full E2E on Fedora Silverblue VM, rpm-ostree rollback, layering conflict handling

4. Arch Linux Support

4.1 pacman-Based Updates

Arch Linux uses pacman as its package manager. Unlike APT and DNF, pacman does not have a sophisticated dependency resolver — it relies on the user running `pacman -Syu` for full system upgrades. This simplicity is actually advantageous for OTA: the entire system is updated as a unit, which is closer to the atomic update model than APT's per-package approach.

Key challenge: Arch is a rolling release. There are no version numbers — only package versions. For OTA, we must create a versioned snapshot of the Arch repository at a known-good state.

4.2 A/B Snapshot Approach Using Btrfs Snapshots

Arch Linux's default filesystem is Btrfs (with the `archinstall` installer offering it as a default). We leverage Btrfs snapshots for atomic updates:

1. Before an update, take a Btrfs snapshot of the root subvolume: `btrfs subvolume snapshot / /snapshots/pre-update-<timestamp>`
2. Run `pacman -Syu` on the live system.
3. If the update succeeds, the snapshot is retained for rollback.
4. If the update fails (or post-update health checks fail), rollback by booting from the snapshot.

package `pacmanadapter`

```
import (
    "context"
    "fmt"
    "os/exec"
    "time"
)
```

```
// ArchAdapter implements OSAdapter for Arch Linux systems.
type ArchAdapter struct {
```

```

    config ArchAdapterConfig
}

// ArchAdapterConfig holds configuration for the Arch Linux
adapter.
type ArchAdapterConfig struct {
    SnapshotDir      string `json:"snapshot_dir"
                        yaml:"snapshot_dir"` // e.g., "/.snapshots"
    RootSubvolume    string `json:"root_subvolume"
                        yaml:"root_subvolume"` // e.g., "@/root"
    CustomRepoURL     string `json:"custom_repo_url"
                        yaml:"custom_repo_url"` // Helix OTA custom repo
    CustomRepoName    string `json:"custom_repo_name"
                        yaml:"custom_repo_name"` // e.g., "helix-ota"
    PackageWhitelist []string `json:"package_whitelist"
                        yaml:"package_whitelist"` // nil = all
    MaxSnapshots      int    `json:"max_snapshots"
                        yaml:"max_snapshots"` // Default: 3
    HealthCheckCmd    string `json:"health_check_cmd"
                        yaml:"health_check_cmd"` // Custom health check script
    Bootloader        string `json:"bootloader"
                        yaml:"bootloader"` // "grub", "systemd-
                        boot"
}

// CheckForUpdate checks for available updates in the custom
repository.
func (a *ArchAdapter) CheckForUpdate(
    ctx context.Context,
    device Device,
) (*UpdateInfo, error) {
    // 1. Sync package databases from Helix custom repository only
    cmd := exec.CommandContext(ctx, "pacman", "-Sy", "--
        noconfirm",
        "--dbpath", "/var/lib/pacman-helix")
    if output, err := cmd.CombinedOutput(); err != nil {
        return nil, fmt.Errorf("pacman sync failed: %s: %w",
            string(output), err)
    }

    // 2. Check for available upgrades (dry run)
    cmd = exec.CommandContext(ctx, "pacman", "-Qu")
    output, err := cmd.CombinedOutput()
    if err != nil {
        // pacman -Qu returns exit code 1 when no updates
available
        return nil, nil
    }

    upgrades := parsePacmanQu(string(output))
    if len(upgrades) == 0 {
        return nil, nil
    }

    return &UpdateInfo{

```

```

        Version:      computeArchFingerprint(upgrades),
        Description:  fmt.Sprintf("%d packages to upgrade",
            len(upgrades)),
        Type:          "pacman_upgrade",
        Metadata:      map[string]interface{}{"packages": upgrades},
    }, nil
}

// ApplyUpdate applies the pacman upgrade with Btrfs snapshot
// protection.
func (a *ArchAdapter) ApplyUpdate(
    ctx context.Context,
    device Device,
    artifact Artifact,
) error {
    // 1. Create pre-update Btrfs snapshot
    snapshotName := fmt.Sprintf("pre-update-%s",
        time.Now().Format("20060102-150405"))
    snapshotPath := fmt.Sprintf("%s/%s", a.config.SnapshotDir,
        snapshotName)
    cmd := exec.CommandContext(ctx, "btrfs", "subvolume",
        "snapshot", "/",
        snapshotPath)
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("btrfs snapshot failed: %s: %w",
            string(output), err)
    }

    // 2. Run pacman upgrade
    args := []string{"-Syu", "--noconfirm", "--overwrite", "*"}
    if len(a.config.PackageWhitelist) > 0 {
        args = append(args, a.config.PackageWhitelist...)
    }
    cmd = exec.CommandContext(ctx, "pacman", args...)
    if output, err := cmd.CombinedOutput(); err != nil {
        // Snapshot is available for rollback
        return fmt.Errorf("pacman upgrade failed: %s: %w",
            string(output), err)
    }

    // 3. Handle .pacnew files
    if err := a.handlePacnewFiles(ctx); err != nil {
        // Non-fatal: log warning but don't fail the update
        // .pacnew files are preserved for manual review
    }

    // 4. Prune old snapshots
    if err := a.pruneSnapshots(ctx); err != nil {
        // Non-fatal: will retry on next update
    }

    return nil
}

```

```

// VerifyUpdate runs health checks after the update.
func (a *ArchAdapter) VerifyUpdate(ctx context.Context, device
    Device) error {
    // 1. Check for broken packages
    cmd := exec.CommandContext(ctx, "pacman", "-Qk")
    if output, err := cmd.CombinedOutput(); err != nil {
        return
            fmt.Errorf("pacman package verification failed: %s: %w",
                string(output), err)
    }

    // 2. Run custom health check if configured
    if a.config.HealthCheckCmd != "" {
        cmd = exec.CommandContext(ctx, "sh", "-c",
            a.config.HealthCheckCmd)
        if output, err := cmd.CombinedOutput(); err != nil {
            return fmt.Errorf("health check failed: %s: %w",
                string(output), err)
        }
    }

    return nil
}

// Rollback reverts to the previous Btrfs snapshot.
func (a *ArchAdapter) Rollback(ctx context.Context, device
    Device) error {
    // 1. Find the most recent pre-update snapshot
    snapshot, err := a.findLatestSnapshot(ctx)
    if err != nil {
        return fmt.Errorf("find snapshot: %w", err)
    }

    // 2. Update bootloader to boot from snapshot
    switch a.config.Bootloader {
    case "grub":
        return a.grubBootSnapshot(ctx, snapshot)
    case "systemd-boot":
        return a.systemdBootSnapshot(ctx, snapshot)
    default:
        return fmt.Errorf("unsupported bootloader: %s",
            a.config.Bootloader)
    }
}

// ReportStatus returns the current update status.
func (a *ArchAdapter) ReportStatus(ctx context.Context, device
    Device) (*UpdateStatus, error) {
    // Query pacman for current package state
    cmd := exec.CommandContext(ctx, "pacman", "-Q")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return nil, fmt.Errorf("pacman query failed: %w", err)
    }
}

```

```

    return &UpdateStatus{
        State: "IDLE",
        Metadata: map[string]interface{}{"pacman_packages":
            string(output)},
    }, nil
}

// handlePacnewFiles manages .pacnew files created during upgrade.
// Default behavior: preserve existing config, keep .pacnew for
// review.
func (a *ArchAdapter) handlePacnewFiles(ctx context.Context)
    error {
    cmd := exec.CommandContext(ctx, "find", "/etc", "-name",
        "*.pacnew")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("find pacnew files: %w", err)
    }
    // Report .pacnew files to server for operator review
    // Do not automatically merge – this is a manual operation
    return nil
}

// pruneSnapshots removes old snapshots exceeding MaxSnapshots.
func (a *ArchAdapter) pruneSnapshots(ctx context.Context) error {
    // List snapshots sorted by creation time
    // Remove oldest if count exceeds MaxSnapshots
    return nil
}

func (a *ArchAdapter) findLatestSnapshot(ctx context.Context)
    (string, error) {
    // Implementation: list and sort snapshots in SnapshotDir
    return "", nil
}

func (a *ArchAdapter) grubBootSnapshot(ctx context.Context,
    snapshot string) error {
    // Update GRUB to boot from the snapshot subvolume
    return nil
}

func (a *ArchAdapter) systemdBootSnapshot(ctx context.Context,
    snapshot string) error {
    // Update systemd-boot entry to point to snapshot subvolume
    return nil
}

```

4.3 Delta Package Support

Arch's pacman supports delta packages natively through `DeltaPackages` in the repository database. We configure our custom repository to generate delta packages:


```
# /etc/pacman.d/repo.conf (custom repo section)
[options]
UseDelta = 0.7 # Use delta if size < 70% of full package
```

On the server side, we generate delta packages using xdelta3:

package pacmanadapter

```
// GeneratePacmanDelta creates a delta package between two package
// versions.
func GeneratePacmanDelta(oldPkg, newPkg string) (string, error) {
    deltaPath := newPkg + ".delta"
    cmd := exec.Command("xdelta3", "-e", "-s", oldPkg, newPkg,
        deltaPath)
    if output, err := cmd.CombinedOutput(); err != nil {
        return "", fmt.Errorf("xdelta3 failed: %s: %w",
            string(output), err)
    }
    return deltaPath, nil
}
```

4.4 Implementation Plan

Phase	Duration	Deliverable
Phase 1: pacman adapter	2 weeks	pacman upgrade/rollback with Btrfs snapshots
Phase 2: Custom repository	1 week	Helix OTA pacman repository hosting with delta packages
Phase 3: .pacnew handling	1 week	Reporting and management of .pacnew configuration files
Phase 4: Testing	1 week	E2E on Arch Linux container with Btrfs, rollback verification

5. Generic Linux A/B Rootfs

5.1 A/B Partition Scheme for Any Linux Distribution

The generic A/B rootfs adapter works with any Linux distribution, regardless of its package manager. It treats the entire root filesystem as a single update unit, identical to Android's approach.

Partition layout:

Partition	Slot A	Slot B
boot	/dev/disk0p1	/dev/disk0p2
rootfs	/dev/disk0p3	/dev/disk0p4

userdata	/dev/disk0p5 (shared)	
swap (optional)	/dev/disk0p6 (shared)	
+-----+-----+-----+		

Update flow:

1. Determine the inactive slot (if current rootfs is on partition 3, target is partition 4).
2. Download the new rootfs image.
3. Write the image to the inactive rootfs partition.
4. Write the corresponding boot partition contents.
5. Update the bootloader to boot from the inactive slot on next reboot.
6. Reboot.
7. On successful boot, mark the new slot as active. On failure, the bootloader falls back to the previous slot.

5.2 GRUB Bootloader Integration for Slot Switching

For x86_64 devices using GRUB, slot switching is achieved by modifying the GRUB environment:

package abrootfs

```
import (
    "context"
    "fmt"
    "os/exec"
)

// GRUBSlotManager manages A/B slot switching via GRUB.
type GRUBSlotManager struct {
    grubEnvFile string // Path to GRUB environment file, e.g., "/boot/grub/grubenv"
}

// SwitchSlot changes the active GRUB boot entry to the specified slot.
func (g *GRUBSlotManager) SwitchSlot(ctx context.Context, slot string) error {
    // Set the GRUB default to the appropriate boot entry
    // GRUB menu entries are named like "Helix OTA (Slot A)" and "Helix OTA (Slot B)"
    entryName := fmt.Sprintf("Helix OTA (Slot %s)", slot)
    cmd := exec.CommandContext(ctx, "grub-set-default", entryName)
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("grub-set-default failed: %s: %w", string(output), err)
    }
    return nil
}

// GetCurrentSlot reads the current active slot from the GRUB environment.
```

```

func (g *GRUBSlotManager) GetCurrentSlot(ctx context.Context)
    (string, error) {
    cmd := exec.CommandContext(ctx, "grub-editenv",
        g.grubEnvFile, "list")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return "", fmt.Errorf("grub-editenv list failed: %s: %w",
            string(output), err)
    }
    return parseGRUBSlot(string(output)), nil
}

// MarkSlotSuccessful marks the current slot as successfully
// booted.
// This prevents the bootloader from falling back on subsequent
// reboots.
func (g *GRUBSlotManager) MarkSlotSuccessful(ctx context.Context,
    slot string) error {
    // In GRUB, we set a "slot_<slot>_boot_success=1" variable
    cmd := exec.CommandContext(ctx, "grub-editenv",
        g.grubEnvFile, "set",
        fmt.Sprintf("slot_%s_boot_success=1", slot))
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("grub-editenv set failed: %s: %w",
            string(output), err)
    }
    return nil
}

```

GRUB configuration template (/etc/grub.d/40_helix_ota):

```

#!/bin/sh
# Helix OTA A/B Boot Entries
cat << EOF
menuentry "Helix OTA (Slot A)" {
    load_env -f /boot/grub/grubenv
    set root=(hd0,gpt1)
    linux /vmlinuz root=/dev/disk0p3 ro helix_slot=_a
    initrd /initramfs.img
    if [ "\$slot_a_boot_success" != "1" ]; then
        # First boot on this slot – if it fails, fall back to Slot
        B
        save_env -f /boot/grub/grubenv slot_a_boot_success=0
    fi
}

menuentry "Helix OTA (Slot B)" {
    load_env -f /boot/grub/grubenv
    set root=(hd0,gpt2)
    linux /vmlinuz root=/dev/disk0p4 ro helix_slot=_b
    initrd /initramfs.img
    if [ "\$slot_b_boot_success" != "1" ]; then
        save_env -f /boot/grub/grubenv slot_b_boot_success=0
    fi
}

```

```
}  
EOF
```

5.3 U-Boot Integration for Embedded Linux

For ARM-based embedded devices (including the Orange Pi 5 Max running Linux instead of Android), U-Boot is the standard bootloader. Slot switching is achieved by modifying U-Boot environment variables:

```
package abrootfs  
  
import (  
    "context"  
    "fmt"  
    "os/exec"  
)  
  
// UBootSlotManager manages A/B slot switching via U-Boot.  
type UBootSlotManager struct {  
    fwEnvConfig string // Path to fw_env.config, e.g., "/etc/  
                        fw_env.config"  
}  
  
// SwitchSlot changes the active U-Boot boot partition.  
func (u *UBootSlotManager) SwitchSlot(ctx context.Context, slot  
    string) error {  
    // U-Boot stores the active slot in environment variables:  
    // boot_slot=_a or boot_slot=_b  
    // The bootcmd script uses these to select the correct  
    // partition.  
    cmd := exec.CommandContext(ctx, "fw_setenv", "boot_slot",  
        slot)  
    if output, err := cmd.CombinedOutput(); err != nil {  
        return fmt.Errorf("fw_setenv failed: %s: %w",  
            string(output), err)  
    }  
  
    // Also set the boot_part variable to indicate the rootfs  
    // partition  
    var partNum string  
    switch slot {  
    case "_a":  
        partNum = "3" // /dev/mmcblk0p3  
    case "_b":  
        partNum = "4" // /dev/mmcblk0p4  
    default:  
        return fmt.Errorf("invalid slot: %s", slot)  
    }  
    cmd = exec.CommandContext(ctx, "fw_setenv", "boot_part",  
        partNum)  
    if output, err := cmd.CombinedOutput(); err != nil {  
        return fmt.Errorf("fw_setenv boot_part failed: %s: %w",  
            string(output), err)  
    }  
}
```

```

    }

    return nil
}

// GetCurrentSlot reads the current active slot from U-Boot
// environment.
func (u *UBootSlotManager) GetCurrentSlot(ctx context.Context)
(string, error) {
    cmd := exec.CommandContext(ctx, "fw_printenv", "boot_slot")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return "", fmt.Errorf("fw_printenv failed: %s: %w",
            string(output), err)
    }
    return parseUBootSlot(string(output)), nil
}

// MarkSlotSuccessful marks the current slot as successfully
// booted.
func (u *UBootSlotManager) MarkSlotSuccessful(ctx
    context.Context, slot string) error {
    // Set slot_<slot>_boot_count to indicate successful boot
    cmd := exec.CommandContext(ctx, "fw_setenv",
        fmt.Sprintf("slot%s_boot_success", slot), "1")
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("fw_setenv failed: %s: %w",
            string(output), err)
    }

    // Reset boot retry counter for the slot
    cmd = exec.CommandContext(ctx, "fw_setenv",
        fmt.Sprintf("slot%s_boot_retries", slot), "0")
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("fw_setenv retries failed: %s: %w",
            string(output), err)
    }

    return nil
}

```

U-Boot boot script (stored as boot.scr):

```

# U-Boot boot script for Helix OTA A/B

# Read active slot from environment
if test "${boot_slot}" = ""; then
    setenv boot_slot _a
fi

# Boot retry logic: if a slot fails to boot 3 times, switch to the
# other slot
if test "${boot_slot}" = "_a"; then
    if test "${slot_a_boot_success}" != "1"; then

```

```

        setexpr slot_a_boot_retries ${slot_a_boot_retries} + 1
        if test ${slot_a_boot_retries} -gt 3; then
            echo
            "Slot A failed to boot 3 times, switching to Slot B"
            setenv boot_slot _b
            setenv slot_a_boot_retries 0
        fi
    fi
else
    if test "${slot_b_boot_success}" != "1"; then
        setexpr slot_b_boot_retries ${slot_b_boot_retries} + 1
        if test ${slot_b_boot_retries} -gt 3; then
            echo
            "Slot B failed to boot 3 times, switching to Slot A"
            setenv boot_slot _a
            setenv slot_b_boot_retries 0
        fi
    fi
fi

# Load kernel and rootfs based on active slot
if test "${boot_slot}" = "_a"; then
    setenv rootpart 3
    setenv bootpart 1
else
    setenv rootpart 4
    setenv bootpart 2
fi

# Reset boot success flag before attempting boot
setenv slot${boot_slot}_boot_success 0
saveenv

# Boot the system
load mmc 0:${bootpart} ${kernel_addr_r} /vmlinuz
load mmc 0:${bootpart} ${fdt_addr_r} /dtb
load mmc 0:${bootpart} ${initrd_addr_r} /initramfs.img
booti ${kernel_addr_r} ${initrd_addr_r} ${fdt_addr_r}

```

5.4 ABRootfsManager Implementation

```
package abrootfs
```

```
import (
    "context"
    "fmt"
    "io"
    "os"
    "os/exec"
)
```

```
// ABRootfsManager manages generic A/B rootfs updates for any
// Linux distribution.
```

```

type ABRootfsManager struct {
    config ABRootfsConfig
    slotMgr SlotManager // Interface for bootloader-specific slot
                        // management
}

// SlotManager abstracts bootloader-specific slot management.
type SlotManager interface {
    SwitchSlot(ctx context.Context, slot string) error
    GetCurrentSlot(ctx context.Context) (string, error)
    MarkSlotSuccessful(ctx context.Context, slot string) error
}

// ABRootfsConfig holds configuration for A/B rootfs updates.
type ABRootfsConfig struct {
    RootfsADevice string `json:"rootfs_a_device"
                        yaml:"rootfs_a_device" // e.g., "/dev/mmcblk0p3"
    RootfsBDevice string `json:"rootfs_b_device"
                        yaml:"rootfs_b_device" // e.g., "/dev/mmcblk0p4"
    BootADevice   string `json:"boot_a_device"
                        yaml:"boot_a_device"   // e.g., "/dev/mmcblk0p1"
    BootBDevice   string `json:"boot_b_device"
                        yaml:"boot_b_device"   // e.g., "/dev/mmcblk0p2"
    UserDataDevice string `json:"userdata_device"
                        yaml:"userdata_device" // e.g., "/dev/mmcblk0p5"
    Bootloader     string `json:"bootloader"
                        yaml:"bootloader"     // "grub", "u-boot",
                        "systemd-boot"
}

// CheckForUpdate determines the inactive slot and prepares for
// update.
func (m *ABRootfsManager) CheckForUpdate(
    ctx context.Context,
    device Device,
) (*UpdateInfo, error) {
    // Verify the A/B partition scheme is intact
    if err := m.validatePartitions(); err != nil {
        return nil, fmt.Errorf("partition validation failed: %w",
            err)
    }
    // The actual update check is delegated to the server
    // This method ensures the device is in a state ready for
    // update
    return nil, nil // Update availability is determined server-
                    // side
}

// ApplyUpdate writes the new rootfs image to the inactive slot.
func (m *ABRootfsManager) ApplyUpdate(
    ctx context.Context,
    device Device,
    artifact Artifact,
) error {

```

```

// 1. Determine inactive slot
currentSlot, err := m.slotMgr.GetCurrentSlot(ctx)
if err != nil {
    return fmt.Errorf("get current slot: %w", err)
}

targetSlot := "_b"
targetRootfs := m.config.RootfsBDevice
targetBoot := m.config.BootBDevice
if currentSlot == "_b" {
    targetSlot = "_a"
    targetRootfs = m.config.RootfsADevice
    targetBoot = m.config.BootADevice
}

// 2. Download the rootfs image (if not already downloaded by
// the SDK)
imagePath := artifact.LocalPath // Set by the download manager

// 3. Verify the image before writing
if err := m.verifyImage(imagePath, artifact.SHA256); err !=
    nil {
    return fmt.Errorf("image verification failed: %w", err)
}

// 4. Write rootfs image to inactive partition
if err := m.writeImageToPartition(ctx, imagePath,
    targetRootfs); err != nil {
    return fmt.Errorf("write rootfs image: %w", err)
}

// 5. Extract and write boot partition contents
if err := m.writeBootPartition(ctx, imagePath, targetBoot);
    err != nil {
    return fmt.Errorf("write boot partition: %w", err)
}

// 6. Switch bootloader to inactive slot
if err := m.slotMgr.SwitchSlot(ctx, targetSlot); err != nil {
    return fmt.Errorf("switch slot: %w", err)
}

return nil
}

// VerifyUpdate confirms the new slot booted successfully.
func (m *ABRootfsManager) VerifyUpdate(ctx
    context.Context, device Device) error {
    currentSlot, err := m.slotMgr.GetCurrentSlot(ctx)
    if err != nil {
        return fmt.Errorf("get current slot: %w", err)
    }

```



```

    // Mark the current slot as successfully booted
    return m.slotMgr.MarkSlotSuccessful(ctx, currentSlot)
}

// Rollback switches back to the previous slot.
func (m *ABRootfsManager) Rollback(ctx context.Context, device
    Device) error {
    currentSlot, err := m.slotMgr.GetCurrentSlot(ctx)
    if err != nil {
        return fmt.Errorf("get current slot: %w", err)
    }

    previousSlot := "_a"
    if currentSlot == "_a" {
        previousSlot = "_b"
    }

    return m.slotMgr.SwitchSlot(ctx, previousSlot)
}

// ReportStatus returns the current A/B rootfs status.
func (m *ABRootfsManager) ReportStatus(ctx
    context.Context, device Device) (*UpdateStatus, error) {
    currentSlot, err := m.slotMgr.GetCurrentSlot(ctx)
    if err != nil {
        return nil, fmt.Errorf("get current slot: %w", err)
    }
    return &UpdateStatus{
        State: "IDLE",
        Metadata: map[string]interface{}{
            "current_slot": currentSlot,
        },
    }, nil
}

// writeImageToPartition writes a disk image to a partition using
// dd.
func (m *ABRootfsManager) writeImageToPartition(
    ctx context.Context,
    imagePath string,
    partitionDevice string,
) error {
    // Open the image file
    imgFile, err := os.Open(imagePath)
    if err != nil {
        return fmt.Errorf("open image: %w", err)
    }
    defer imgFile.Close()

    // Open the target partition
    partFile, err := os.OpenFile(partitionDevice, os.O_WRONLY, 0)
    if err != nil {
        return fmt.Errorf("open partition: %w", err)
    }

```

```

    }
    defer partFile.Close()

    // Stream the image to the partition (with progress reporting)
    written, err := io.Copy(partFile, imgFile)
    if err != nil {
        return fmt.Errorf("write image: %w (wrote %d bytes)",
            err, written)
    }

    // Sync to ensure all data is flushed to disk
    if err := partFile.Sync(); err != nil {
        return fmt.Errorf("sync partition: %w", err)
    }

    return nil
}

// verifyImage checks the SHA-256 hash of the rootfs image.
func (m *ABRootfsManager) verifyImage(imagePath, expectedSHA256
    string) error {
    // Use the helix-ota-client-sdk's verifier package
    // This reuses the same SHA-256 verification as Android
    return nil // Delegated to the SDK verifier
}

// validatePartitions verifies that the A/B partition scheme
// exists.
func (m *ABRootfsManager) validatePartitions() error {
    for _, dev := range []string{
        m.config.RootfsADevice, m.config.RootfsBDevice,
        m.config.BootADevice, m.config.BootBDevice,
    } {
        if _, err := os.Stat(dev); err != nil {
            return fmt.Errorf("partition %s not found: %w", dev,
                err)
        }
    }
    return nil
}

func (m *ABRootfsManager) writeBootPartition(ctx context.Context,
    imagePath, bootDevice string) error {
    // Extract boot contents from the rootfs image and write to
    // boot partition
    // This is distribution-specific; the default implementation
    // assumes
    // the boot partition is a separate FAT/ext2 partition
    // containing
    // vmlinuz, initramfs, and DTB files
    return nil
}

```

5.5 Implementation Plan

Phase	Duration	Deliverable
Phase 1: Core ABRootfsManager	2 weeks	Generic A/B rootfs update with partition writing
Phase 2: GRUB integration	1 week	GRUB slot switching with fallback logic
Phase 3: U-Boot integration	2 weeks	U-Boot slot switching for RK3588 and other ARM platforms
Phase 4: systemd-boot integration	1 week	systemd-boot slot switching for modern x86_64 systems
Phase 5: Testing	2 weeks	E2E on Debian x86_64 (GRUB), Debian ARM64 (U-Boot), Fedora (systemd-boot)

6. OSAdapter Plugin Interface Design

6.1 Go Interface Definition

The OSAdapter interface is the central abstraction that makes Helix OTA multi-OS. It was first sketched in the v1.0.0 architecture document as a forward-looking design principle, and v1.1.0 is where it becomes concrete.

```
package osadapter
```

```
import (  
    "context"  
    "time"  
)  
  
// OSAdapter defines the contract for OS-specific update logic.  
// Every supported OS (Android, Ubuntu, Fedora, Arch, Windows)  
// implements  
// this interface. The server and client SDK communicate with OS-  
// specific  
// logic exclusively through this interface.  
type OSAdapter interface {  
    // Name returns the human-readable name of the adapter.  
    // Example: "ubuntu-apt", "fedora-rpm-ostree", "arch-pacman",  
    // "linux-ab-rootfs"  
    Name() string  
  
    // OSType returns the OS type identifier used in API  
    // communication.  
    // Example: "linux/ubuntu", "linux/fedora", "linux/arch",  
    // "android"  
    OSType() string
```

```

// Capabilities returns the set of update capabilities this
// adapter supports.
// This allows the server to tailor update artifacts to what
// the device can handle.
Capabilities() AdapterCapabilities

// CheckForUpdate queries the device's current state and
// determines
// whether an update is available for this OS type.
CheckForUpdate(ctx context.Context, device DeviceInfo)
(*UpdateInfo, error)

// PrepareUpdate prepares the device for an update (e.g.,
// creating snapshots,
// freeing disk space, checking prerequisites).
PrepareUpdate(ctx context.Context, device
DeviceInfo, artifact ArtifactInfo) error

// ApplyUpdate triggers the OS-specific update mechanism.
ApplyUpdate(ctx context.Context, device DeviceInfo, artifact
ArtifactInfo) error

// VerifyUpdate confirms the update was applied correctly.
// Called after reboot for A/B updates, or after apply for in-
// place updates.
VerifyUpdate(ctx context.Context, device DeviceInfo) error

// CommitUpdate marks the update as permanent, preventing
// automatic rollback.
// For A/B systems, this marks the current slot as successful.
CommitUpdate(ctx context.Context, device DeviceInfo) error

// Rollback reverts to the previous system state.
Rollback(ctx context.Context, device DeviceInfo) error

// ReportStatus returns the current update status.
ReportStatus(ctx context.Context, device DeviceInfo)
(*UpdateStatus, error)

// HealthCheck verifies the device is in a healthy state for
// updates.
HealthCheck(ctx context.Context, device DeviceInfo)
(*HealthReport, error)
}

// AdapterCapabilities describes what an adapter can do.
type AdapterCapabilities struct {
    SupportsAtomicUpdate bool
    `json:"supports_atomic_update"` // Can update atomically
    (A/B, OSTree)
    SupportsDeltaUpdate bool
    `json:"supports_delta_update"` // Can apply delta/
    differential updates

```

```

    SupportsRollback      bool
        `json:"supports_rollback"`      // Can rollback to
        previous state
    SupportsPackageLevel  bool
        `json:"supports_package_level"`  // Can update individual
        packages
    SupportsFullImage     bool
        `json:"supports_full_image"`     // Can replace entire
        rootfs
    RequiresReboot        bool
        `json:"requires_reboot"`         // Update requires a
        reboot to take effect
    SupportsOfflineUpdate bool
        `json:"supports_offline_update"` // Can prepare update
        offline
    MaxSnapshotCount      int
        `json:"max_snapshot_count"`      // Number of rollback
        snapshots retained
}

// DeviceInfo contains device information relevant to update
// decisions.
type DeviceInfo struct {
    DeviceID      string `json:"device_id"`
    CurrentVersion string `json:"current_version"`
    OSType        string `json:"os_type"`
    OSVersion     string `json:"os_version"`
    HardwareModel string `json:"hardware_model"`
    CurrentSlot   string
        `json:"current_slot,omitempty"`
    InstalledPackages []PackageInfo
        `json:"installed_packages,omitempty"`
    FreeDiskBytes  int64 `json:"free_disk_bytes"`
    BatteryPercent int
        `json:"battery_percent,omitempty"`
    IsOnACPower    bool
        `json:"is_on_ac_power,omitempty"`
    Uptime         time.Duration `json:"uptime"`
    LastRebootTime time.Time    `json:"last_reboot_time"`
    CustomMetadata map[string]string
        `json:"custom_metadata,omitempty"`
}

// PackageInfo describes an installed package.
type PackageInfo struct {
    Name      string `json:"name"`
    Version   string `json:"version"`
    Arch      string `json:"arch"`
}

// ArtifactInfo contains information about an update artifact.
type ArtifactInfo struct {
    ArtifactID  string `json:"artifact_id"`
    Version     string `json:"version"`
    DownloadURL string `json:"download_url"`
}

```

```

    SHA256            string            `json:"sha256"`
    SizeBytes         int64             `json:"size_bytes"`
    SignatureURL      string            `json:"signature_url"`
    UpdateType        string            `json:"update_type"` //
        "full", "delta", "apt_upgrade", "pacman_upgrade"
    Metadata          map[string]interface{} `json:"metadata"`
    LocalPath         string            `json:"local_path,omitempty"` // Set by download manager
}

```

// UpdateInfo describes an available update.

```

type UpdateInfo struct {
    Available    bool            `json:"available"`
    Version      string          `json:"version"`
    Description  string          `json:"description"`
    Type         string          `json:"type"`
    SizeBytes    int64          `json:"size_bytes"`
    Mandatory    bool           `json:"mandatory"`
    Deadline     *time.Time     `json:"deadline,omitempty"`
    Metadata     map[string]interface{} `json:"metadata,omitempty"`
}

```

// UpdateStatus represents the current update state of a device.

```

type UpdateStatus struct {
    State        string          `json:"state"` // IDLE,
        CHECKING, DOWNLOADING, VERIFYING, PREPARING, APPLYING,
        REBOOTING, COMMITTING, SUCCESS, FAILED, ROLLING_BACK
    ProgressPercent int            `json:"progress_percent"`
    ErrorCode      string          `json:"error_code,omitempty"`
    ErrorMessage   string          `json:"error_message,omitempty"`
    CurrentSlot    string          `json:"current_slot,omitempty"`
    Metadata       map[string]interface{} `json:"metadata,omitempty"`
    Timestamp      time.Time       `json:"timestamp"`
}

```

// HealthReport describes the health status of a device.

```

type HealthReport struct {
    Healthy      bool            `json:"healthy"`
    Issues       []string        `json:"issues,omitempty"`
    DiskUsageBytes int64          `json:"disk_usage_bytes"`
    DiskTotalBytes int64          `json:"disk_total_bytes"`
    MemoryUsedBytes int64          `json:"memory_used_bytes"`
    MemoryTotalBytes int64          `json:"memory_total_bytes"`
    Uptime       time.Duration   `json:"uptime"`
    Metadata     map[string]interface{} `json:"metadata,omitempty"`
}

```

6.2 Plugin Loading Mechanism

For v1.1.0, we use a compile-time plugin mechanism (factory pattern). Dynamic plugin loading (Go plugins or WASM) is deferred to v2.0.0 as specified in the roadmap.

```
package osadapter

import (
    "fmt"
    "sync"
)

// Registry holds all registered OSAdapter implementations.
type Registry struct {
    mu      sync.RWMutex
    adapters map[string]OSAdapter
}

// globalRegistry is the default adapter registry.
var globalRegistry = &Registry{
    adapters: make(map[string]OSAdapter),
}

// Register adds an adapter to the global registry.
// This is typically called from an init() function in each
// adapter package.
func Register(adapter OSAdapter) {
    globalRegistry.Register(adapter)
}

// Get returns an adapter by OS type.
func Get(osType string) (OSAdapter, error) {
    return globalRegistry.Get(osType)
}

// Register adds an adapter to this registry.
func (r *Registry) Register(adapter OSAdapter) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.adapters[adapter.OSType()] = adapter
}

// Get returns an adapter by OS type.
func (r *Registry) Get(osType string) (OSAdapter, error) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    adapter, ok := r.adapters[osType]
    if !ok {
        return nil,
            fmt.Errorf("no adapter registered for OS type: %s",
                osType)
    }
}
```

```

    return adapter, nil
}

// List returns all registered adapters.
func (r *Registry) List() []OSAdapter {
    r.mu.RLock()
    defer r.mu.RUnlock()
    result := make([]OSAdapter, 0, len(r.adapters))
    for _, adapter := range r.adapters {
        result = append(result, adapter)
    }
    return result
}

// --- Adapter registration via init() ---

// In each adapter package, register the adapter:

// helix-apt-adapter/adapter.go:
// func init() {
//     osadapter.Register(&UbuntuAdapter{})
// }

// helix-ostree-adapter/adapter.go:
// func init() {
//     osadapter.Register(&RpmOstreeAdapter{})
//     osadapter.Register(&BootcAdapter{})
// }

// helix-pacman-adapter/adapter.go:
// func init() {
//     osadapter.Register(&ArchAdapter{})
// }

```

6.3 Per-Distribution Adapter Implementations

Adapter	Package	OSType	Capabilities
Ubuntu APT	helix-apt-adapter	linux/ ubuntu	AtomicUpdate: no (APT mode), yes (A/B mode); DeltaUpdate: yes; Rollback: yes (A/B mode); PackageLevel: yes (APT mode); FullImage: yes (A/B mode); RequiresReboot: yes (A/B mode), no (APT mode)
Fedora rpm-ostree	helix-ostree-adapter	linux/ fedora	AtomicUpdate: yes; DeltaUpdate: yes (static deltas); Rollback: yes; PackageLevel: yes

Adapter	Package	OSType	Capabilities
Fedora bootc	helix-ostree-adapter	linux/fedora-bootc	(layering); FullImage: yes; RequiresReboot: yes AtomicUpdate: yes; DeltaUpdate: yes (OCI layer diffs); Rollback: yes; PackageLevel: no; FullImage: yes; RequiresReboot: yes
Arch pacman	helix-pacman-adapter	linux/arch	AtomicUpdate: yes (Btrfs); DeltaUpdate: yes (xdelta3); Rollback: yes (Btrfs); PackageLevel: yes; FullImage: no; RequiresReboot: no
Generic A/B	helix-linux-client	linux/generic-ab	AtomicUpdate: yes; DeltaUpdate: yes (bsdiff); Rollback: yes; PackageLevel: no; FullImage: yes; RequiresReboot: yes
Android	(existing)	android	AtomicUpdate: yes; DeltaUpdate: yes; Rollback: yes; PackageLevel: no; FullImage: yes; RequiresReboot: yes

6.4 Configuration Schema

Each adapter's configuration is stored in a YAML file on the device and registered with the Helix Linux client:

```
# /etc/helix-ota/adapter.yaml
adapter:
  os_type: "linux/ubuntu"
  mode: "ab_rootfs" # or "apt"

# Common settings
server_url: "https://ota.example.com"
device_id: "dev_01HXYZ..."
check_interval: "4h"

# A/B rootfs settings (when mode=ab_rootfs)
ab_rootfs:
  rootfs_a_device: "/dev/mmcblk0p3"
  rootfs_b_device: "/dev/mmcblk0p4"
  boot_a_device: "/dev/mmcblk0p1"
  boot_b_device: "/dev/mmcblk0p2"
  userdata_device: "/dev/mmcblk0p5"
  bootloader: "u-boot" # or "grub", "systemd-boot"
```

```
# APT settings (when mode=apt)
apt:
  repo_url: "https://ota.example.com/apt"
  repo_key_url: "https://ota.example.com/apt/gpg-key.asc"
  distro: "ubuntu"
  codename: "noble"
  component: "helix-ota"
  package_whitelist:
    - "linux-image-*"
    - "systemd"
    - "openssl"

# Security settings
security:
  public_key_pem: "/etc/helix-ota/public.pem"
  ca_cert_pem: "/etc/helix-ota/ca.pem"
  device_cert_pem: "/etc/helix-ota/device.pem"
  device_key_pem: "/etc/helix-ota/device.key"
```

6.5 Testing Strategy Per Adapter

Each adapter must pass the following test matrix:

Test Category	Ubuntu	Fedora (rpm-ostree)	Arch	Generic A/B
Unit: CheckForUpdate	APT dry-run parsing	rpm-ostree status parsing	pacman -Qu parsing	Partition validation
Unit: ApplyUpdate	APT upgrade execution	rpm-ostree upgrade execution	pacman -Syu with Btrfs snapshot	dd image to partition
Unit: Rollback	A/B slot switch	rpm-ostree rollback	Btrfs snapshot boot	Slot switch
Unit: VerifyUpdate	dpkg audit	rpm-ostree status	pacman -Qk	Boot verification
Integration	Ubuntu 24.04 container	Fedora Silverblue VM	Arch container + Btrfs	Debian x86_64 + GRUB
Mutation	85% score	85% score	85% score	85% score
E2E	Full A/B cycle on RK3588 Debian	Full upgrade on Silverblue VM	Full upgrade + rollback on Arch	Full cycle on RK3588 + U-Boot
Chaos				

Test Category	Ubuntu	Fedora (rpm-ostree)	Arch	Generic A/B
	Kill apt mid-upgrade	Kill rpm-ostree mid-apply	Kill pacman mid-upgrade	Power loss during dd write

7. New Submodules Required

7.1 helix-linux-client: Linux OTA Client Daemon

Property	Value
Repository	HelixDevelopment/helix-linux-client
Go module	dev.helix.ota.linuxclient
Language	Go 1.22+
Target platforms	linux/amd64, linux/arm64
License	Apache 2.0

The `helix-linux-client` is a `systemd` daemon that runs on Linux devices and manages the OTA update lifecycle. It embeds the `helix-ota-client-sdk` for server communication and uses the `OSAdapter` interface for OS-specific update operations.

Package structure:

```
helix-linux-client/
├── cmd/
│   └── helix-ota-client/
│       └── main.go                # Daemon entry point
├── internal/
│   ├── daemon/
│   │   ├── daemon.go            # Main daemon loop
│   │   ├── daemon_test.go
│   │   └── systemd.go           # systemd notification
├── integration
│   ├── signals.go                # Signal handling
│   ├── config/
│   │   ├── config.go            # Configuration loading
│   │   └── config_test.go
│   ├── scheduler/
│   │   ├── scheduler.go         # Update check scheduling
│   │   ├── scheduler_test.go
│   │   └── jitter.go            # Jitter for check interval
│   └── health/
│       ├── checker.go           # Post-update health check
│       └── checker_test.go
└── pkg/
    └── adapter/
```

```

├── loader.go          # Adapter registry and loading
├── contrib/
│   ├── systemd/
│   │   └── helix-ota-client.service # systemd unit file
│   └── init/
│       └── helix-ota-client.init    # SysV init script
├── go.mod
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md

```

Key daemon implementation:

```
package daemon
```

```
import (
    "context"
    "os/signal"
    "syscall"
    "time"

    "dev.helix.ota.linuxclient/internal/scheduler"
    "dev.helix.ota.linuxclient/internal/health"
    "dev.helix.ota.clientsdk/pkg/clientsdk"
    "dev.helix.ota.osadapter"
)
```

```
// Daemon is the main OTA client daemon.
```

```
type Daemon struct {
    sdk      clientsdk.HelixOtaSDK
    adapter  osadapter.OSAdapter
    scheduler *scheduler.Scheduler
    health   *health.Checker
    config   DaemonConfig
}
```

```
// DaemonConfig holds the daemon configuration.
```

```
type DaemonConfig struct {
    ServerURL      string      `yaml:"server_url"`
    DeviceID       string      `yaml:"device_id"`
    CheckInterval  time.Duration `yaml:"check_interval"`
    OSType         string      `yaml:"os_type"`
    AdapterConfig  string      `yaml:"adapter_config" // Path
                        to adapter YAML
    HealthCheckCmd string      `yaml:"health_check_cmd"`
    AutoReboot     bool       `yaml:"auto_reboot"`
    RebootDelay    time.Duration `yaml:"reboot_delay"`
}
```

```
// Run starts the daemon's main loop.
```

```
func (d *Daemon) Run(ctx context.Context) error {
```

```

ctx, stop := signal.NotifyContext(ctx, syscall.SIGINT,
    syscall.SIGTERM)
defer stop()

// Notify systemd that we're ready
notifySystemd("READY=1")

// Start the update check scheduler
checkCh := d.scheduler.Start(ctx)

for {
    select {
    case <-checkCh:
        if err := d.performUpdateCheck(ctx); err != nil {
            d.sdk.ReportStatus(ctx, clientsdk.UpdateStatus{
                State:      "FAILED",
                ErrorCode: "CHECK_FAILED",
                ErrorMessage: err.Error(),
            })
        }
    case <-ctx.Done():
        notifySystemd("STOPPING=1")
        return ctx.Err()
    }
}

}

// performUpdateCheck checks for and applies an update.
func (d *Daemon) performUpdateCheck(ctx context.Context) error {
    // 1. Check for update
    updateInfo, err := d.adapter.CheckForUpdate(ctx,
        d.getDeviceInfo())
    if err != nil {
        return fmt.Errorf("check for update: %w", err)
    }
    if updateInfo == nil || !updateInfo.Available {
        return nil // no update available
    }

    // 2. Download artifact via SDK
    artifact, err := d.sdk.DownloadUpdate(ctx,
        &clientsdk.UpdateInfo{
            ArtifactID: updateInfo.Version,
            DownloadURL: updateInfo.Metadata["download_url"].(string),
            SHA256:      updateInfo.Metadata["sha256"].(string),
        }, nil)
    if err != nil {
        return fmt.Errorf("download update: %w", err)
    }

    // 3. Prepare for update
    if err := d.adapter.PrepareUpdate(ctx, d.getDeviceInfo(),
        artifactToArtifactInfo(artifact)); err != nil {

```

```

        return fmt.Errorf("prepare update: %w", err)
    }

    // 4. Apply update
    d.sdk.ReportStatus(ctx, clientsdk.UpdateStatus{State:
        "APPLYING"})
    if err := d.adapter.ApplyUpdate(ctx, d.getDeviceInfo(),
        artifactToArtifactInfo(artifact)); err != nil {
        d.sdk.ReportStatus(ctx, clientsdk.UpdateStatus{State:
            "FAILED", ErrorMessage: err.Error()})
        return fmt.Errorf("apply update: %w", err)
    }

    // 5. Verify update
    if err := d.adapter.VerifyUpdate(ctx, d.getDeviceInfo());
        err != nil {
        // Rollback on verification failure
        d.adapter.Rollback(ctx, d.getDeviceInfo())
        return fmt.Errorf("verify update: %w", err)
    }

    // 6. Run health check
    report, err := d.health.Check(ctx)
    if err != nil || !report.Healthy {
        d.adapter.Rollback(ctx, d.getDeviceInfo())
        return fmt.Errorf("health check failed")
    }

    // 7. Commit update
    if err := d.adapter.CommitUpdate(ctx, d.getDeviceInfo());
        err != nil {
        return fmt.Errorf("commit update: %w", err)
    }

    // 8. Reboot if required
    if d.adapter.Capabilities().RequiresReboot &&
        d.config.AutoReboot {
        time.AfterFunc(d.config.RebootDelay, func() {
            syscall.Reboot(syscall.LINUX_REBOOT_CMD_RESTART)
        })
    }

    d.sdk.ReportStatus(ctx, clientsdk.UpdateStatus{State:
        "SUCCESS"})
    return nil
}

```

7.2 helix-ostree-adapter: OSTree Integration

Property	Value
Repository	HelixDevelopment/helix-ostree-adapter
Go module	dev.helix.ota.ostreeadapter

Property	Value
Language	Go 1.22+
License	Apache 2.0

This submodule provides both client-side and server-side OSTree integration. On the client side, it implements the `OSAdapter` interface for `rpm-ostree` and `bootc`. On the server side, it manages OSTree repositories, generates static deltas, and serves content via S3.

Package structure:

```

helix-ostree-adapter/
├── client/
│   ├── rpm_ostree_adapter.go    # OSAdapter implementation for
rpm-ostree
│   └── bootc_adapter.go        # OSAdapter implementation for
bootc
├── rpm_ostree_adapter_test.go
├── bootc_adapter_test.go
├── server/
│   ├── repository.go           # OSTree repository management
│   ├── delta.go                # Static delta generation
│   ├── signing.go              # GPG commit signing
│   ├── repository_test.go
│   ├── delta_test.go
│   └── signing_test.go
├── pkg/
│   ├── commit/
│   │   ├── commit.go           # OSTree commit parsing and types
│   │   └── commit_test.go
│   └── delta/
│       ├── delta.go            # Delta format types
│       └── delta_test.go
├── go.mod
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md

```

7.3 helix-apt-adapter: APT Repository Management

Property	Value
Repository	HelixDevelopment/helix-apt-adapter
Go module	dev.helix.ota.aptadapter
Language	Go 1.22+
License	Apache 2.0

This submodule provides both client-side and server-side APT integration. On the client side, it implements the `OSAdapter` interface for Ubuntu/Debian APT updates. On the server side, it manages APT repositories with GPG signing.

Package structure:

```
helix-apt-adapter/
├── client/
│   ├── ubuntu_adapter.go      # OSAdapter implementation for
│   └── ubuntu_adapter_test.go  Ubuntu/Debian
├── server/
│   ├── repository.go          # APT repository management
│   └── index.go                # Packages/Release index
├── generation
│   ├── signing.go              # GPG repository signing
│   ├── repository_test.go
│   ├── index_test.go
│   └── signing_test.go
├── pkg/
│   ├── deb/
│   │   ├── control.go          # .deb control file parsing
│   │   └── control_test.go
│   └── repo/
│       ├── structure.go         # Repository directory structure
│       └── structure_test.go
├── go.mod
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md
```

7.4 helix-pacman-adapter: Arch Linux Support

Property	Value
Repository	HelixDevelopment/helix-pacman-adapter
Go module	dev.helix.ota.pacmanadapter
Language	Go 1.22+
License	Apache 2.0

This submodule provides both client-side and server-side pacman integration. On the client side, it implements the `OSAdapter` interface for Arch Linux with Btrfs snapshot support. On the server side, it manages custom pacman repositories with delta package generation.

Package structure:

```
helix-pacman-adapter/
├── client/
│   └── arch_adapter.go        # OSAdapter implementation for
```



```

Arch Linux
├── arch_adapter_test.go
├── btrfs.go                # Btrfs snapshot management
├── btrfs_test.go
├── pacnew.go               # .pacnew file handling
├── server/
│   ├── repository.go      # pacman repository management
│   └── delta.go           # xdelta3 delta package
├── generation
│   ├── repository_test.go
│   └── delta_test.go
├── pkg/
│   ├── pkginfo/
│   │   ├── pkginfo.go    # .PKGINFO file parsing
│   │   └── pkginfo_test.go
│   └── alpm/
│       ├── db.go         # ALPM database parsing
│       └── db_test.go
├── go.mod
├── Makefile
├── README.md
├── CLAUDE.md
├── AGENTS.md
└── CONTRIBUTING.md

```

8. Migration Path from Android-Only

8.1 How to Extend Existing Server for Multi-OS

The Helix OTA server (v1.0.0) was designed from the start with the “Universal” principle: the Android adapter is the first implementation of the OSAdapter interface, not a special case. Extending the server for Linux support requires changes at three levels:

Level 1: Data Model Extensions

The existing Device model must be extended to support Linux-specific fields:

```

-- Migration: V005__add_linux_device_support.sql

-- Add OS type column to devices table
ALTER TABLE devices ADD COLUMN os_type VARCHAR(50) NOT NULL
    DEFAULT 'android';

-- Add Linux-specific device metadata
ALTER TABLE devices ADD COLUMN os_version VARCHAR(100);
ALTER TABLE devices ADD COLUMN kernel_version VARCHAR(100);
ALTER TABLE devices ADD COLUMN installed_packages JSONB;
ALTER TABLE devices ADD COLUMN adapter_config JSONB;

-- Add artifact type support

```

```

ALTER TABLE artifacts ADD COLUMN artifact_type VARCHAR(50) NOT
    NULL DEFAULT 'android_ota';
-- Possible values: 'android_ota', 'linux_rootfs', 'apt_package',
    'ostree_commit', 'pacman_package', 'bootc_image'

-- Add artifact metadata for Linux
ALTER TABLE artifacts ADD COLUMN os_type VARCHAR(50);
ALTER TABLE artifacts ADD COLUMN architecture VARCHAR(20);
ALTER TABLE artifacts ADD COLUMN repository_metadata JSONB;
-- For APT: {"distro": "ubuntu", "codename": "noble", "component":
    "helix-ota"}
-- For OSTree: {"branch": "fedora/40/x86_64/silverblue", "commit":
    "abc123..."}
-- For pacman: {"repo_name": "helix-ota", "arch": "x86_64"}

-- Update device groups to include OS type
ALTER TABLE device_groups ADD COLUMN os_type VARCHAR(50) NOT NULL
    DEFAULT 'android';

```

Level 2: Service Layer Extensions

The UpdateService.CheckForUpdate method must be made OS-aware:

package service

```

// CheckForUpdate determines whether an update is available for a
    device.
// This method is now OS-aware: it delegates to the appropriate
    adapter
// for OS-specific compatibility checks.
func (s *UpdateService) CheckForUpdate(
    ctx context.Context,
    deviceID string,
) (*UpdateInfo, error) {
    device, err := s.deviceRepo.GetByID(ctx, deviceID)
    if err != nil {
        return nil, fmt.Errorf("device lookup: %w", err)
    }

    // Find active rollout targeting this device's group and OS
    type
    rollout, err := s.rolloutRepo.FindActiveForGroupAndOS(ctx,
        device.Group, device.OSType)
    if err != nil {
        return nil, fmt.Errorf("rollout lookup: %w", err)
    }
    if rollout == nil {
        return nil, nil
    }

    // Deterministic cohort assignment (unchanged from v1.0.0)
    cohort := fnv32Hash(deviceID) % 100
    if cohort >= rollout.CurrentPercentage {
        return nil, nil
    }
}

```

```

}

// Version compatibility check is now OS-aware
artifact, err := s.artifactRepo.GetByID(ctx,
    rollout.ArtifactID)
if err != nil {
    return nil, fmt.Errorf("artifact lookup: %w", err)
}

// Verify artifact OS type matches device OS type
if artifact.OSType != device.OSType {
    return nil, nil // wrong OS type
}

return &UpdateInfo{
    ArtifactID:  artifact.ID,
    Version:     artifact.TargetVersion,
    DownloadURL: fmt.Sprintf("/api/v1/artifacts/%s/download", artifact.ID),
    SHA256:      artifact.SHA256,
    SizeBytes:   artifact.SizeBytes,
    SignatureURL: fmt.Sprintf("/api/v1/artifacts/%s/signature", artifact.ID),
    Metadata: UpdateMetadata{
        Mandatory: rollout.Mandatory,
        Deadline:   rollout.Deadline,
        ArtifactType: artifact.ArtifactType,
        OSVersion:  artifact.RepositoryMetadata,
    },
}, nil
}

```

Level 3: API Extensions

New API endpoints are added for Linux-specific operations while existing endpoints remain backward-compatible:

Method	Path	New in v1.1.0	Description
POST	/api/v1/devices/register	Extended	Registration now accepts <code>os_type</code> , <code>os_version</code> , <code>kernel_version</code>
GET	/api/v1/devices/{id}/update-check	Extended	Response includes <code>artifact_type</code> and OS-specific metadata
POST	/api/v1/artifacts/upload	Extended	Upload now accepts <code>os_type</code> , <code>architecture</code> , <code>artifact_type</code> , <code>repository_metadata</code>
GET		New	Filter devices by OS type

Method	Path	New in v1.1.0	Description
	/api/v1/ devices? os_type=linux		
GET	/api/v1/ artifacts? os_type=linux	New	Filter artifacts by OS type
POST	/api/v1/ ostree/ commits	New	Publish an OSTree commit
GET	/api/v1/ ostree/ commits/ {branch}	New	List commits on a branch
POST	/api/v1/ ostree/deltas	New	Generate a static delta
POST	/api/v1/apt/ publish	New	Publish packages to APT repository
POST	/api/v1/ pacman/ publish	New	Publish packages to pacman repository

8.2 Dashboard Changes for Linux Device Management

The dashboard must be extended to support multi-OS device management:

New UI components:

1. **OS Type Filter:** A dropdown/filter on the Devices page to filter by OS type (Android, Ubuntu, Fedora, Arch, etc.).
2. **OS-Specific Device Detail:** The device detail page shows OS-specific information (package list for APT/pacman, OSTree commit for rpm-ostree, slot status for A/B).
3. **Multi-OS Artifact Upload:** The artifact upload page now supports multiple artifact types with OS-specific metadata fields.
4. **OS-Aware Rollout Creation:** The rollout creation wizard includes an OS type selector and shows only artifacts matching the selected OS.
5. **Linux-Specific Fleet Overview:** A fleet overview that shows device counts and update status broken down by OS type and distribution version.

Dashboard API type extensions:

// types/api.ts additions for v1.1.0

```
export interface Device {
  device_id: string;
  status: "registered" | "online" | "offline" | "decommissioned";
  hardware_model: string;
  firmware_version: string;
  os_type: "android" | "linux/ubuntu" | "linux/fedora" | "linux/arch" | "linux/generic-ab";
}
```

```

os_version?:
    string;           // Linux-specific: e.g., "24.04", "40",
                      "rolling"
kernel_version?: string;           // Linux-specific: e.g.,
                      "6.8.0-45-generic"
current_slot?: string;           // A/B slot: "_a" or "_b"
installed_packages?: PackageInfo[]; // Linux-specific: list of
                      installed packages
adapter_config?: Record<string, unknown>; // Linux-specific:
                      adapter configuration
last_seen: string;
}

export interface Artifact {
    artifact_id: string;
    version: string;
    hardware_models: string[];
    os_type: string;
    artifact_type: "android_ota" | "linux_rootfs" | "apt_package" |
                  "ostree_commit" | "pacman_package" | "bootc_image";
    architecture?: string;           // Linux-specific: "amd64",
                      "arm64"
    size_bytes: number;
    sha256: string;
    validation_status: "passed" | "failed" | "pending";
    repository_metadata?: Record<string, unknown>; // OS-specific
                      repository info
    created_at: string;
}

```

8.3 API Versioning Strategy

The API versioning follows a **compatible extension** model:

1. **Existing endpoints remain unchanged.** All v1.0.0 API endpoints work identically in v1.1.0. No breaking changes.
2. **New fields are optional.** The `os_type` field defaults to "android" for backward compatibility. Devices registered with v1.0.0 clients continue to work without modification.
3. **New endpoints are additive.** The `/api/v1/ostree/*`, `/api/v1/apt/*`, and `/api/v1/pacman/*` endpoints are new and do not affect existing clients.
4. **API version header:** Clients send `X-Helix-API-Version: 1.1` to opt into v1.1.0 features. Without this header, the server responds with v1.0.0-compatible data (e.g., omitting Linux-specific fields).

package middleware

```

// APIVersionMiddleware extracts the API version from the request
// header
// and injects it into the context.
func APIVersionMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        version := r.Header.Get("X-Helix-API-Version")

```

```

    if version == "" {
        version = "1.0" // default to v1.0.0 compatibility
    }
    ctx := context.WithValue(r.Context(), apiVersionKey{},
        version)
    next.ServeHTTP(w, r.WithContext(ctx))
})
}

```

Version compatibility matrix:

Client Version	Server v1.0.0	Server v1.1.0	Server v2.0.0
Android client v1.0.0	✓ Full support	✓ Full support	✓ Full support
Linux client v1.1.0	✗ Not supported	✓ Full support	✓ Full support
Windows client v1.2.0	✗ Not supported	✗ Not supported	✓ Full support

8.4 Migration Checklist

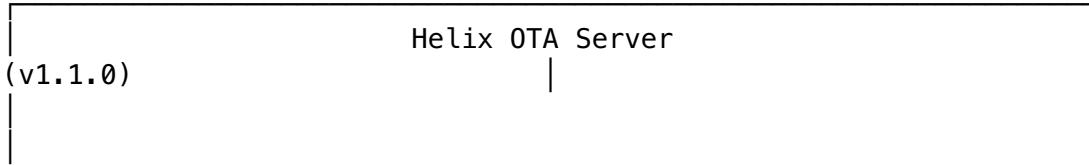
Step	Description	Impact
1	Run database migration V005	Adds os_type, os_version, kernel_version columns with defaults
2	Deploy server v1.1.0	Backward-compatible; Android devices continue to work
3	Enable Linux device registration	Set os_type field on new device registrations
4	Create Linux device groups	Separate groups for Ubuntu, Fedora, Arch, generic A/B
5	Set up APT repository hosting	Configure S3 bucket, GPG key, and repository structure
6	Set up OSTree repository hosting	Configure S3 bucket, GPG key, and static delta generation
7	Deploy helix-linux-client to Linux devices	Install daemon, configure adapter, start systemd service
8	Create first Linux rollout	Select Linux artifact, target Linux device group, start phased rollout

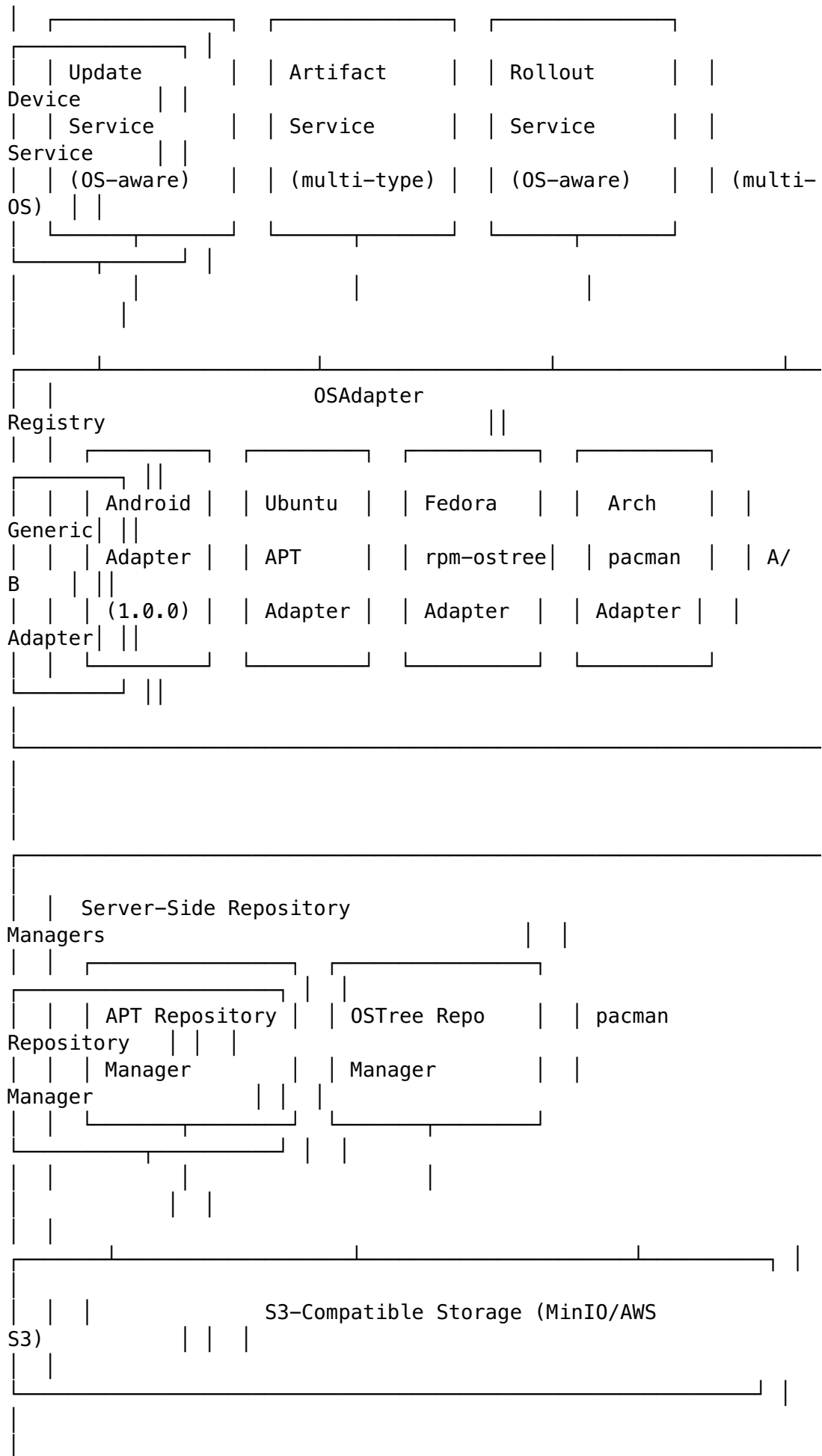
Step	Description	Impact
9	Monitor Linux fleet	Use dashboard OS type filters, verify update success rates
10	Validate rollback for Linux devices	Trigger rollback, verify device returns to previous state

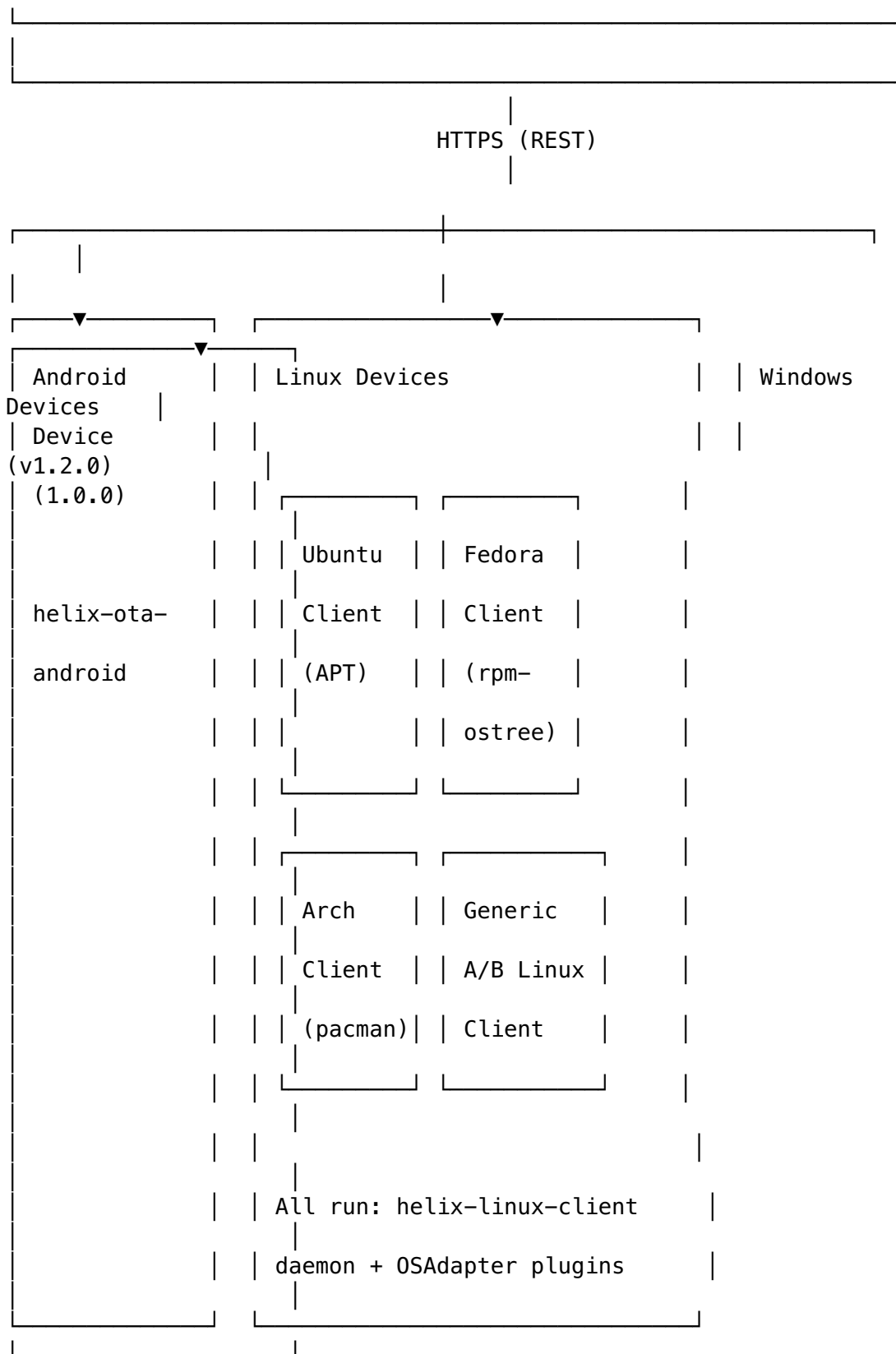
Appendix A — Glossary

Term	Definition
A/B Rootfs	A partition scheme with two root filesystem partitions, allowing atomic updates by writing to the inactive partition
bootc	Bootable Container; a system that boots from OCI container images, the successor to rpm-ostree
Btrfs	B-tree File System; a copy-on-write filesystem supporting snapshots, used for Arch Linux rollback
debdelta	A tool for generating binary diffs between consecutive .deb package versions
deltarpm	RPM delta package format; natively supported by DNF
GRUB	Grand Unified Bootloader; the standard bootloader for x86_64 Linux
OSTree	A content-addressed filesystem tree manager (“git for OS binaries”)
pacman	Arch Linux’s package manager
rpm-ostree	A hybrid of RPM and OSTree providing atomic updates with package layering
Static Delta	A pre-computed binary diff between two OSTree commits for efficient download
U-Boot	Universal Bootloader; the standard bootloader for ARM/embedded Linux
unattended-upgrades	Ubuntu’s automatic security update daemon
xdelta3	A binary diff algorithm used for pacman delta packages

Appendix B — Reference Architecture Diagram







This document is a living artifact. As the Linux OTA implementation progresses through the phases outlined above, each section should be updated with implementation notes, test results, and lessons learned. All code examples are illustrative and must be refined during implementation to meet the 85% mutation testing threshold required by HelixConstitution §1.1.